南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

计算机组成原理
COMPUTER ORGANIZATION

This PowerPoint is for internal use only at Southern University of Science and Technology. Please do not repost it on other platforms without permission from the instructor.

zhangj4@sustech.edu.cn

Lecture 6

Arithmetic Operations on Integers

zhangj4@sustech.edu.cn

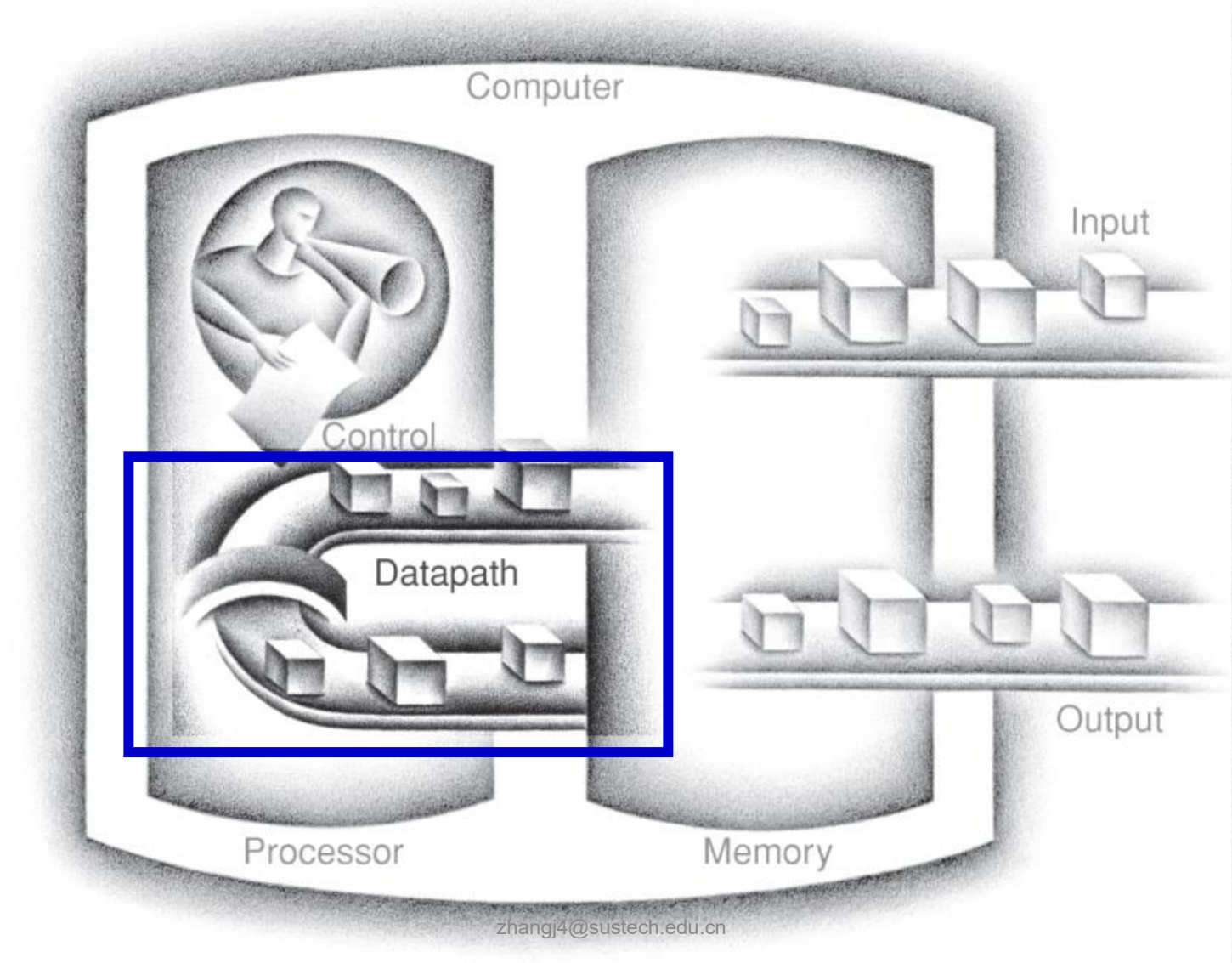
Recap

- Computer Performance
  - Response time vs. throughput
  - Performance = 1 / Execution Time
  - Notions: clock frequency (f) / clock period (Tc) / clock cycle
- Execution Time
  - Elapsed time
  - CPU time = IC ✕ CPI ✕ Tc = IC ✕ CPI / f
  - Notions: instruction count (IC) / cycles per instruction (CPI)
- Power consumption
  - Power = ½ ✕ C ✕ V^2 ✕ f
- Performance comparison
  - SPEC CPU benchmark, geometric mean
- Amdahl's law: $T_{improved}=T_{affected}/factor + T_{unaffected}$

Outline

- Integer addition
- Integer subtraction
- Integer multiplication
- Integer division

Datapath

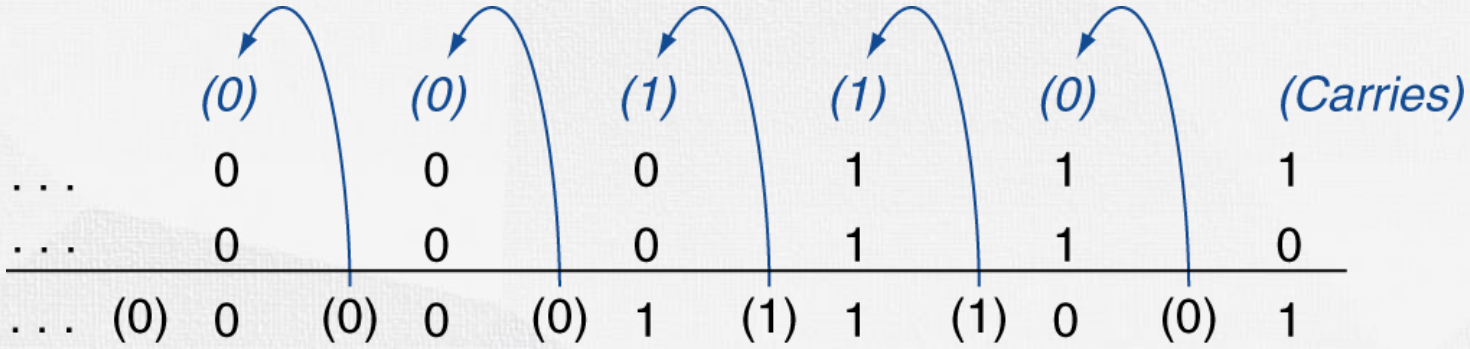


Integer Addition

zhangj4@sustech.edu.cn

Integer Addition

- Example: 7 + 6

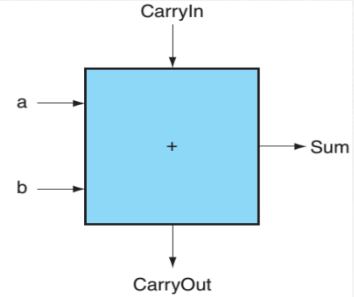


- Overflow
  - please write down the 8-bit signed integer addition:
  - $0100\ 0000_{two}+0100\ 0000_{two}=?$
  - $1000\ 0000_{two}+1000\ 0000_{two}=?$
  - $0100\ 0000_{two}+1100\ 0000_{two}=?$
  - $1100\ 0000_{two}+1100\ 0000_{two}=?$
  - Please write down the equations in binary and decimal.

Overflow

- Examples in last page:
  - 8-bit signed integer range: -128 ~ 127
  - $0100\ 0000_{two}+0100\ 0000_{two}=1000\ 0000_{two}$ 64 + 64= -128 Overflow
  - $1000\ 0000_{two}+1000\ 0000_{two}=(1)0000\ 0000_{two}$ -128 + (-128) = 0 Overflow
  - $0100\ 0000_{two}+1100\ 0000_{two}=(1)0000\ 0000_{two}$ 64+ (-64)= 0 No overflow
  - $1100\ 0000_{two}+1100\ 0000_{two}=(1)1000\ 0000_{two}$ -64+(-64)=-128 No overflow
- Overflow if result out of range
  - no overflow, if ddding +ve(positive) and –ve(negative) operands
  - Overflow, if:
    - Adding two +ve operands, get –ve operand
    - Adding two -ve operands, get +ve operand

1-bit adder



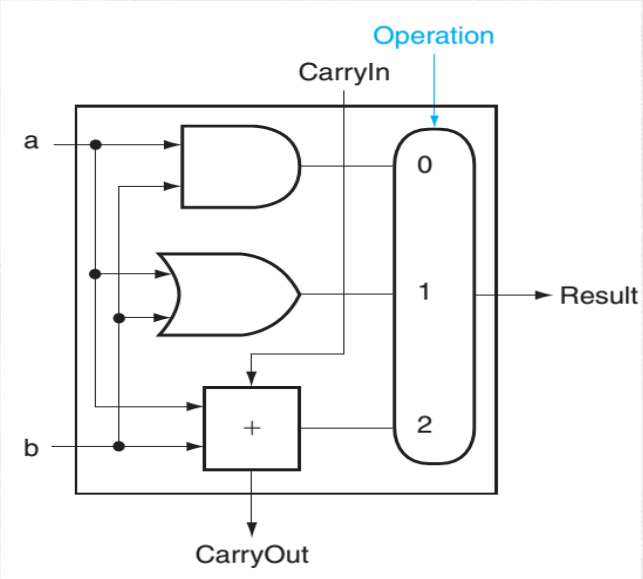
$$\text{Sum} = (a \cdot \overline{b} \cdot \overline{\text{CarryIn}}) + (\overline{a} \cdot b \cdot \overline{\text{CarryIn}}) + (\overline{a} \cdot \overline{b} \cdot \text{CarryIn}) + (a \cdot b \cdot \text{CarryIn})$$

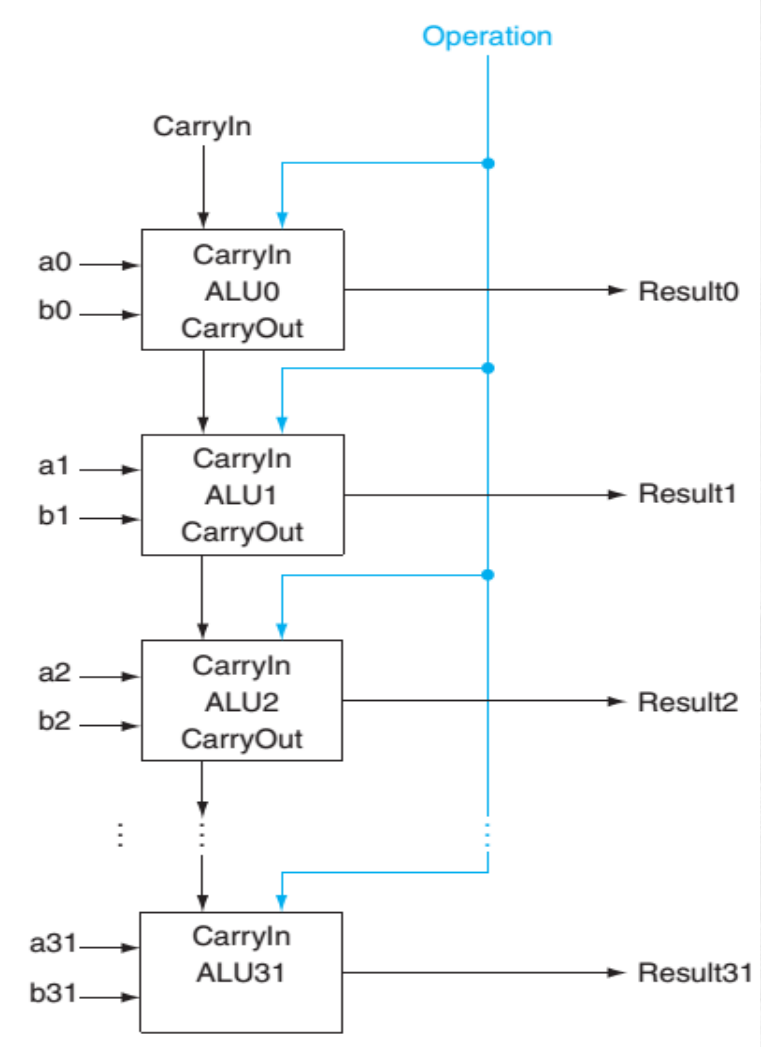
$$\text{CarryOut} = (b \cdot \text{CarryIn}) + (a \cdot \text{CarryIn}) + (a \cdot b)$$

Inputs			Outputs		
a	b	CarryIn	CarryOut	Sum	Comments
0	0	0	0	0	$0 + 0 + 0 = 00_{two}$
0	0	1	0	1	$0 + 0 + 1 = 01_{two}$
0	1	0	0	1	$0 + 1 + 0 = 01_{two}$
0	1	1	1	0	$0 + 1 + 1 = 10_{two}$
1	0	0	0	1	$1 + 0 + 0 = 01_{two}$
1	0	1	1	0	$1 + 0 + 1 = 10_{two}$
1	1	0	1	0	$1 + 1 + 0 = 10_{two}$
1	1	1	1	1	$1 + 1 + 1 = 11_{two}$

1-bit ALU and 32-bit ALU

- ALU: arithmetic logical unit
- 1-bit ALU and 32-bit ALU
  - If op = 0, o = a & b (and)
  - If op = 1, o = a | b (or)
  - If op = 2, o = a + b (add)





Integer Subtraction

zhangj4@sustech.edu.cn

Integer Subtraction

- Add negation of second operand
- Example: 7 – 6 = 7 + (–6)

+7:	0000 0000 … 0000 0111
–6:	1111 1111 … 1111 1010
+1:	0000 0000 … 0000 0001

- Overflow if result out of range
  - No overflow, if subtracting two +ve or two –ve operands
  - Overflow, if:
    - Subtracting +ve from –ve operand, and the result sign is 0 (+ve)
    - Subtracting –ve from +ve operand, and the result sign is 1 (-ve)

Overflow Situations

Operation	Operand A	Operand B	Result indicating overflow
$A + B$	≥ 0	≥ 0	< 0
$A + B$	< 0	< 0	≥ 0
$A - B$	≥ 0	< 0	< 0
$A - B$	< 0	≥ 0	≥ 0

- Example:
  - 12+3=15 120+15=135 12-3=9 -100-50=-150

    12-3 = 12+(-3) -100-50=-100+(-50)

```
  00001100      01111000      00001100      10011100
+ 00000011    + 00001111    + 11111101    + 11001110
  --------      --------      --------      --------
  00001111      10000111      00001001      01101010
```

Overflow

Arithmetic for Multimedia – Saturating Operation

- Graphics and media processing operates on vectors of 8-bit and 16-bit data
  - Use 64-bit adder, with partitioned carry chain
    - Operate on 8×8-bit, 4×16-bit, or 2×32-bit vectors
  - SIMD (single-instruction, multiple-data)
- Saturating operations
  - On overflow, result is largest representable value
    - c.f. 2s-complement modulo arithmetic
    - E.g. a=200, b=200, both are 8-bit unsigned integers (range: 0-255)
    - For regular addition: a+b=400 mod 256 = 144, for saturating addition: a+b=255
  - E.g., change the volume and brightness in audio or video

Integer Multiplication

zhangj4@sustech.edu.cn

Multiplication

- In every step
  - multiplicand is shifted
  - next bit of multiplier is examined (also a shifting step)
  - if this bit is 1, shifted multiplicand is added to the product

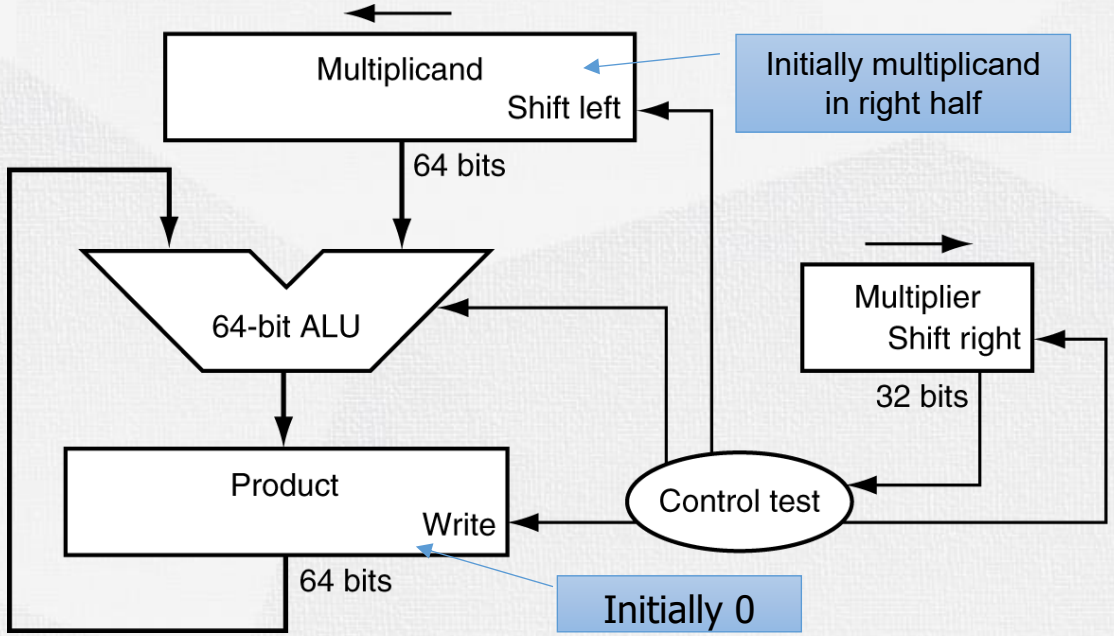
multiplicand

multiplier

product

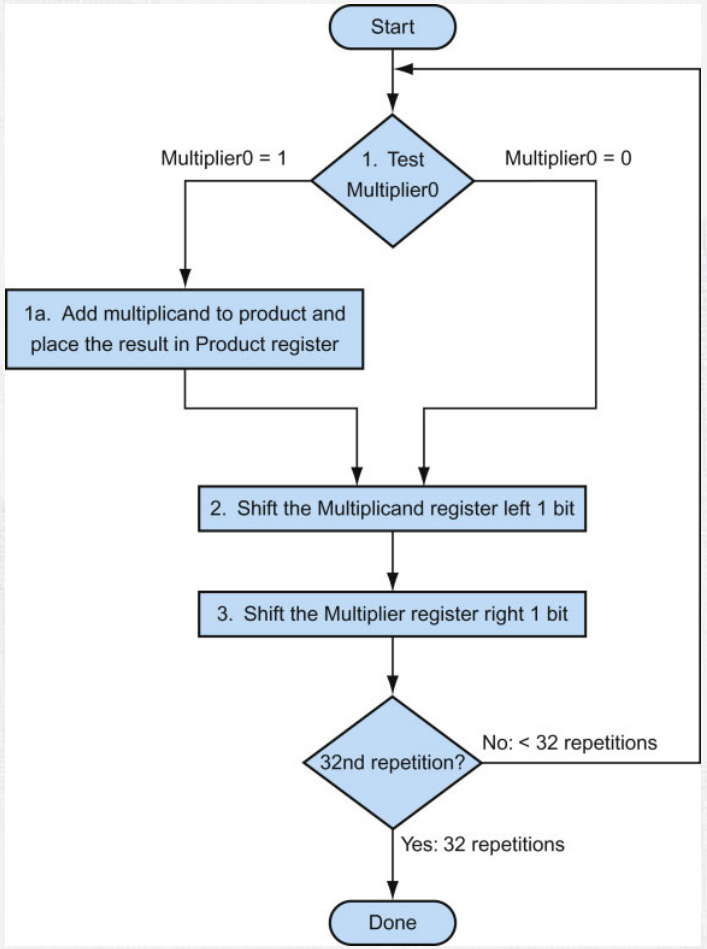
```
      1000
  ×   1001
  --------
      1000
     0000
    0000
   1000
  --------
   1001000
```

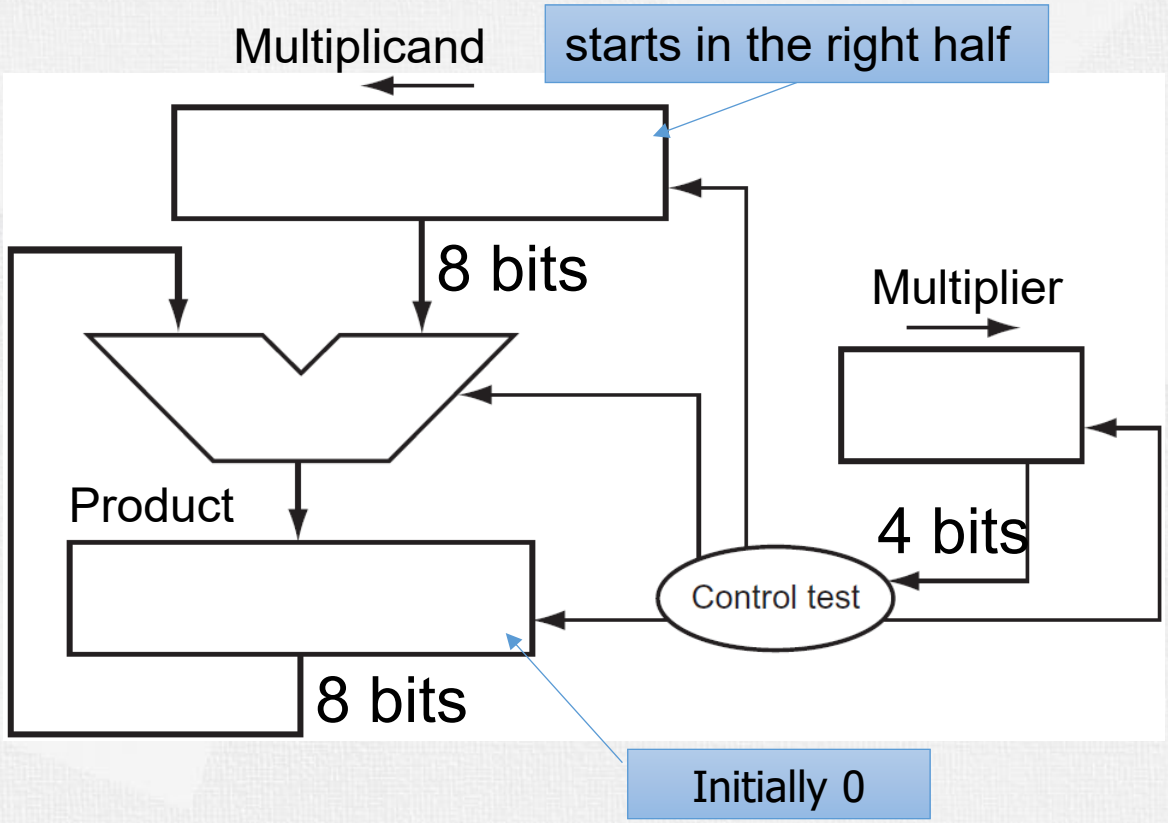
Length of product is the sum of operand lengths



Multiplication Example

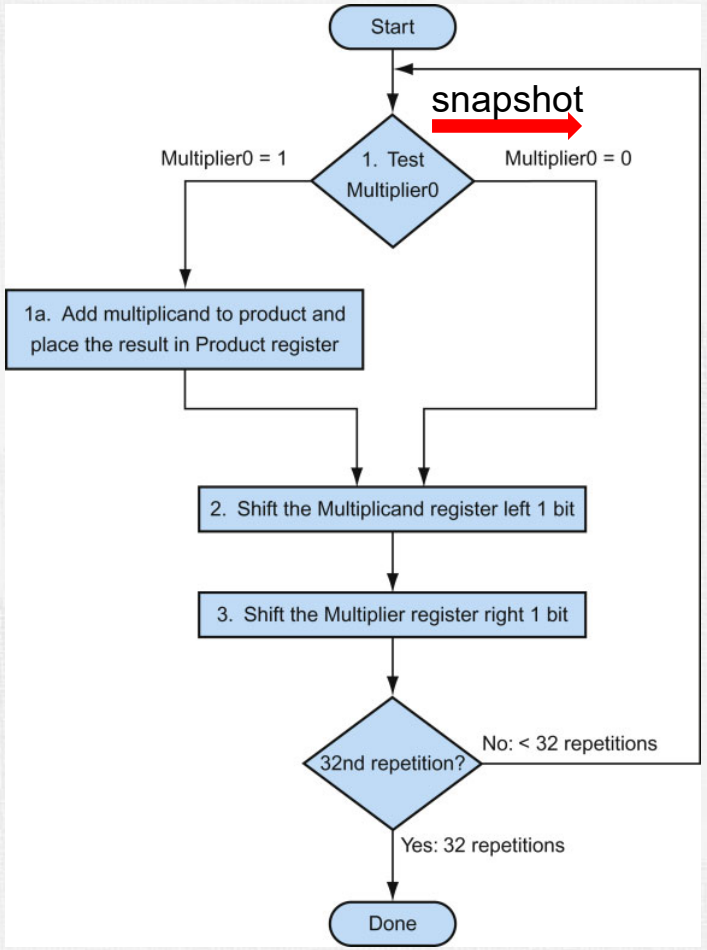
- Multiply 8_{ten} (1000_{two}) by 9_{ten} (1001_{two})
  - How values change in Mcand, Mplier and Product Registers?

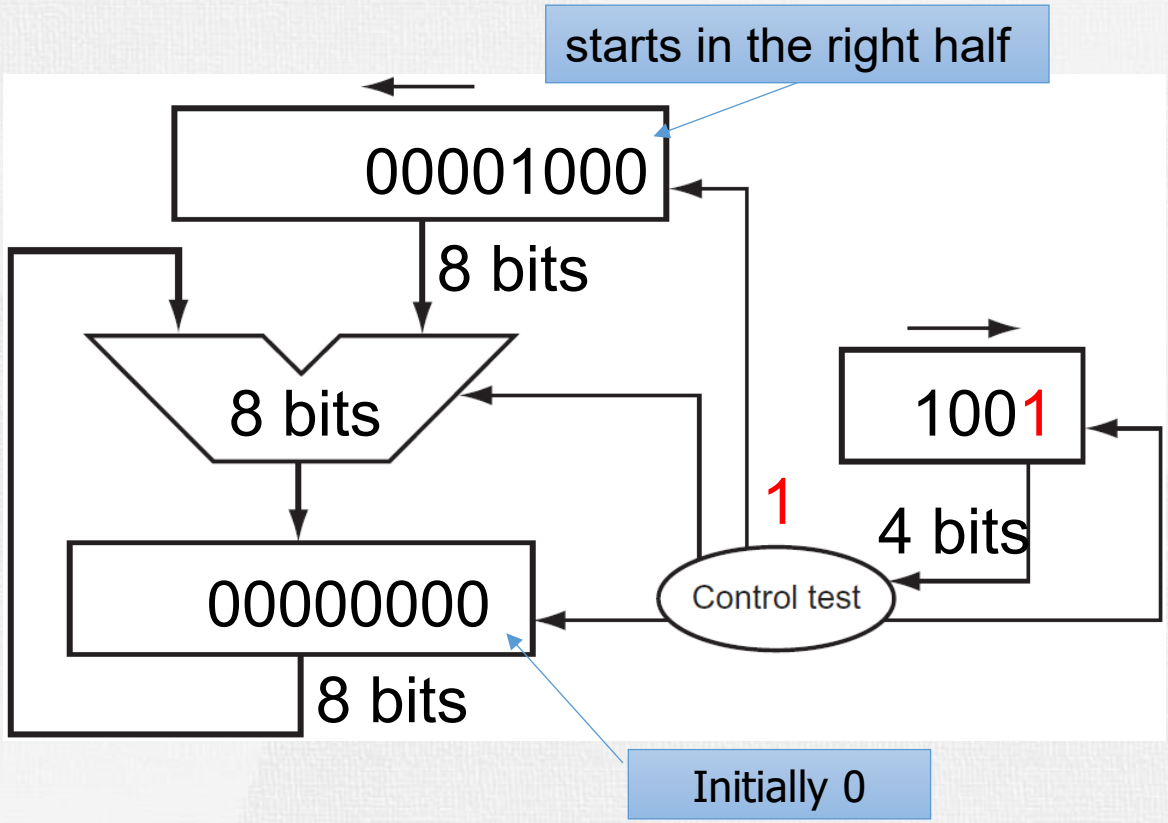




Multiplication Example

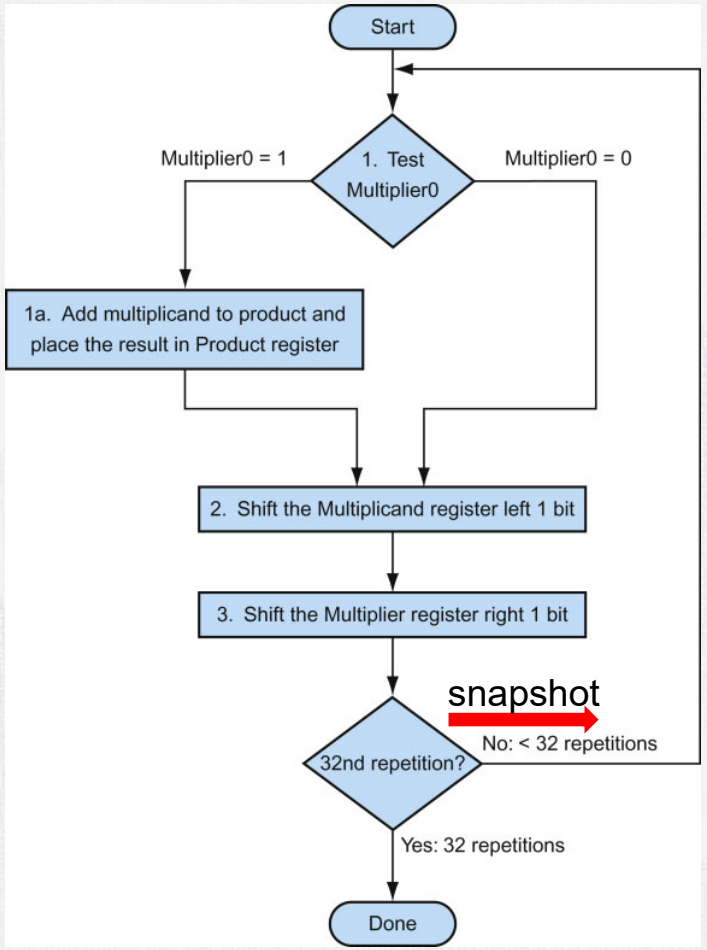
- Multiply 8_{ten} (1000_{two}) by 9_{ten} (1001_{two})

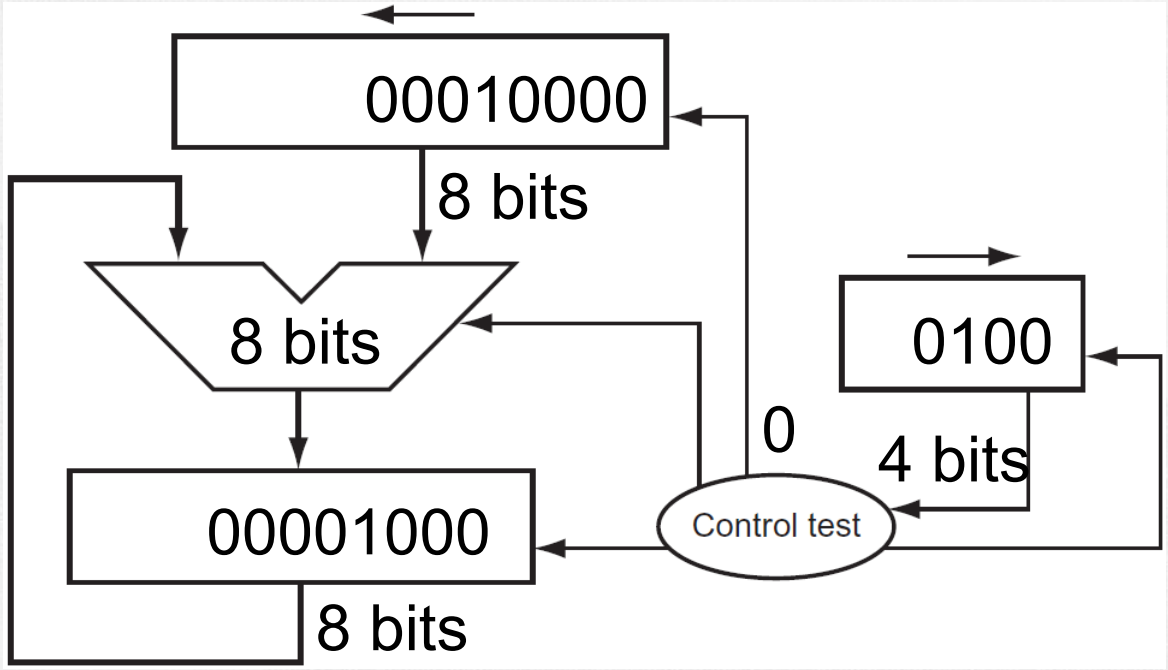




Multiplication Example

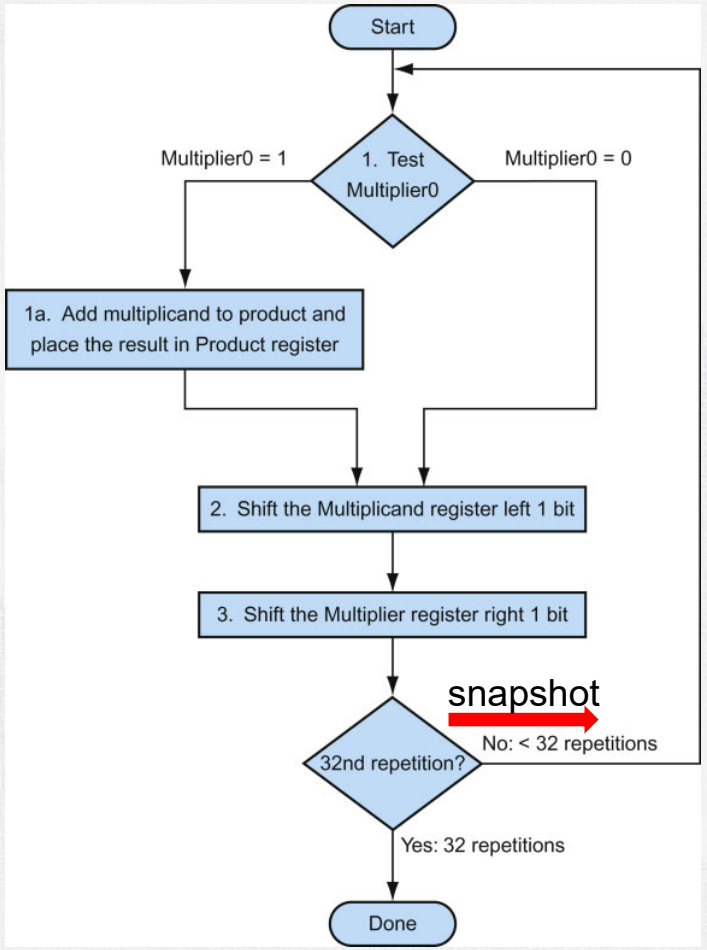
- Multiply 8_{ten} (1000_{two}) by 9_{ten} (1001_{two})

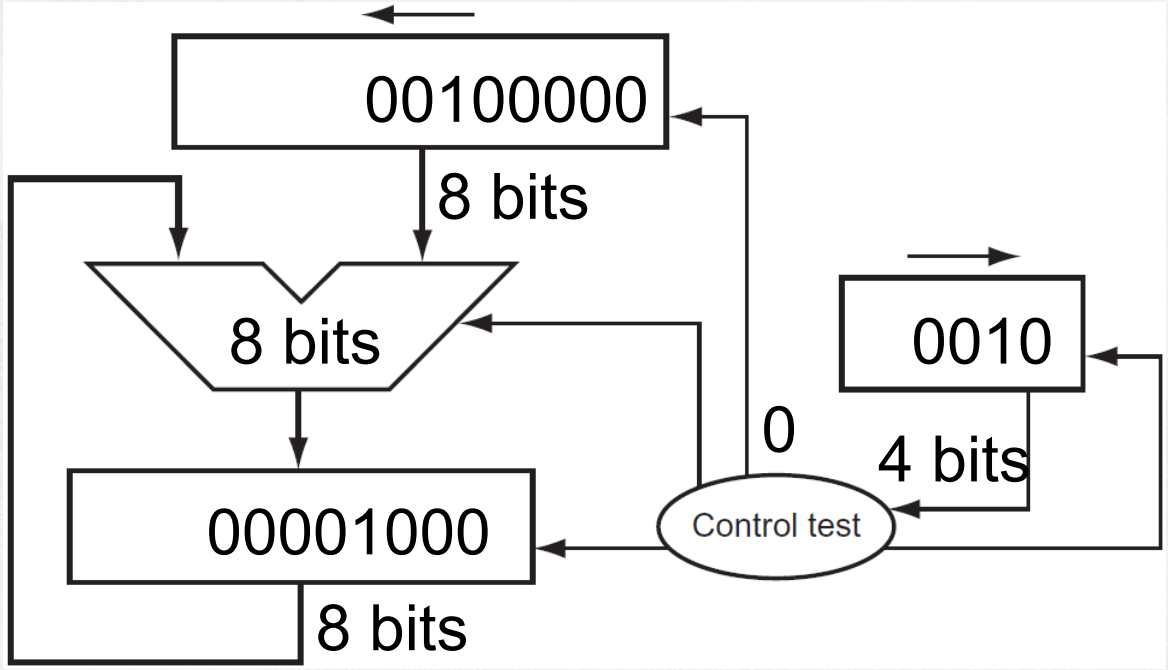




Multiplication Example

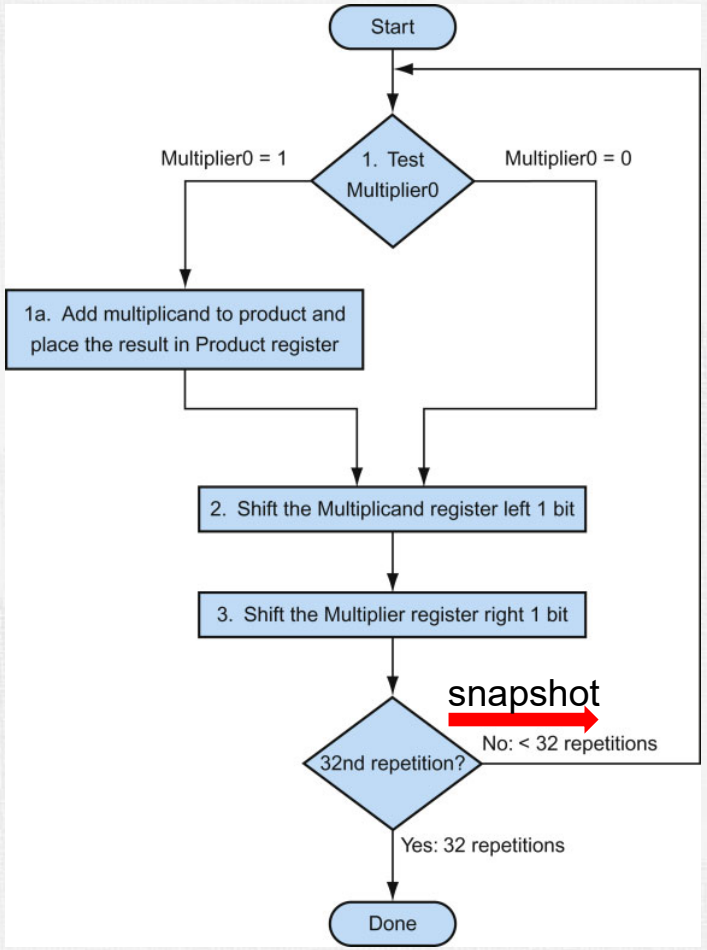
- Multiply 8_{ten} (1000_{two}) by 9_{ten} (1001_{two})

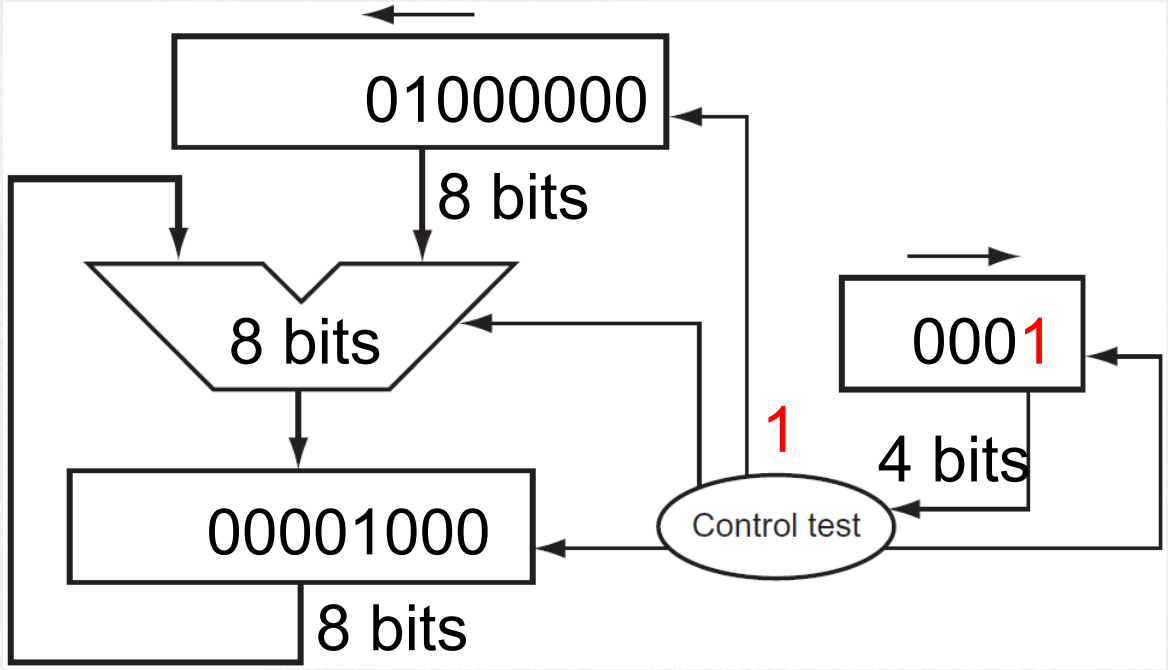




Multiplication Example

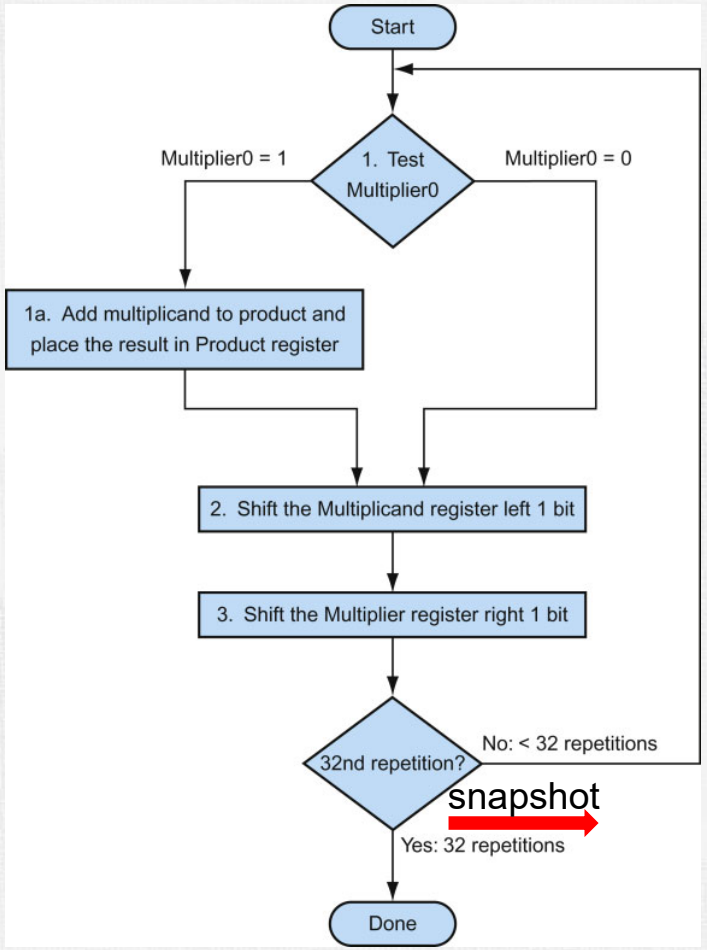
- Multiply 8_{ten} (1000_{two}) by 9_{ten} (1001_{two})

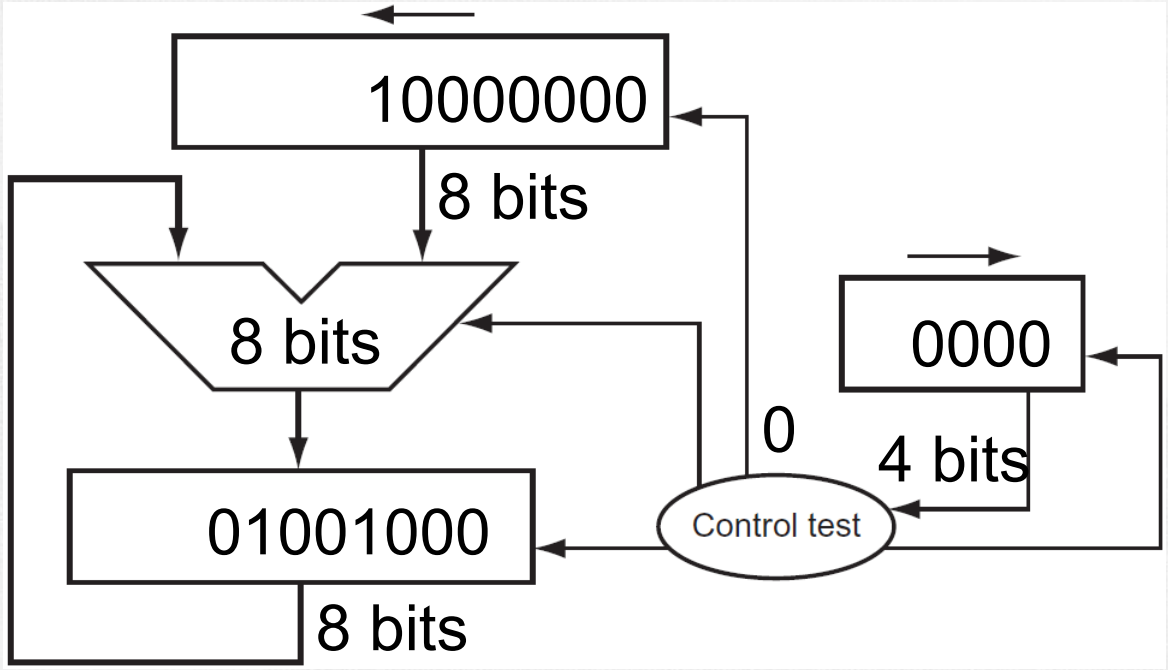




Multiplication Example

- Multiply 8_{ten} (1000_{two}) by 9_{ten} (1001_{two})





Done!

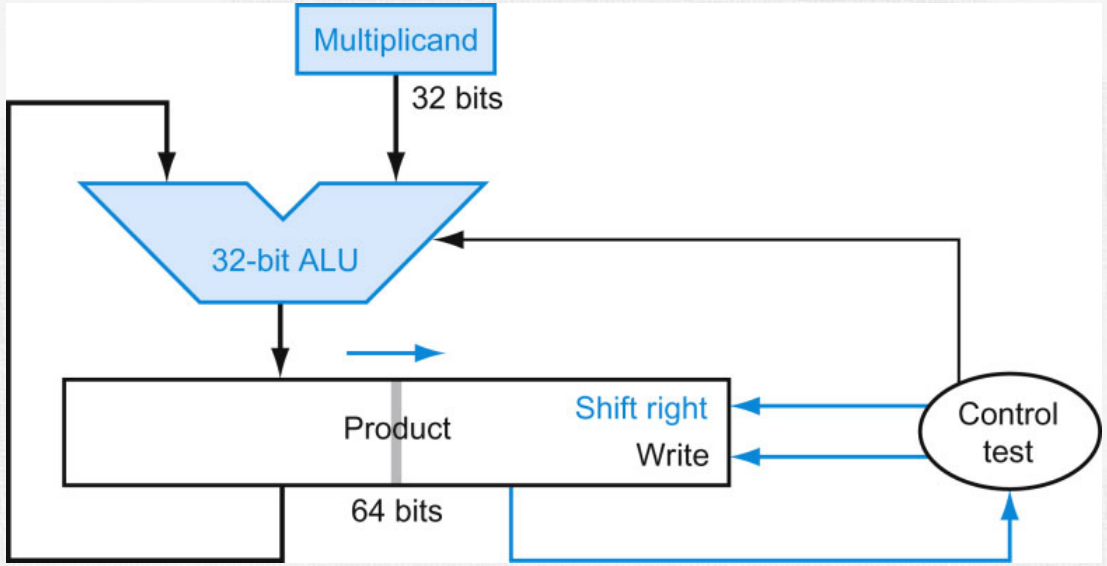
Multiplication Example

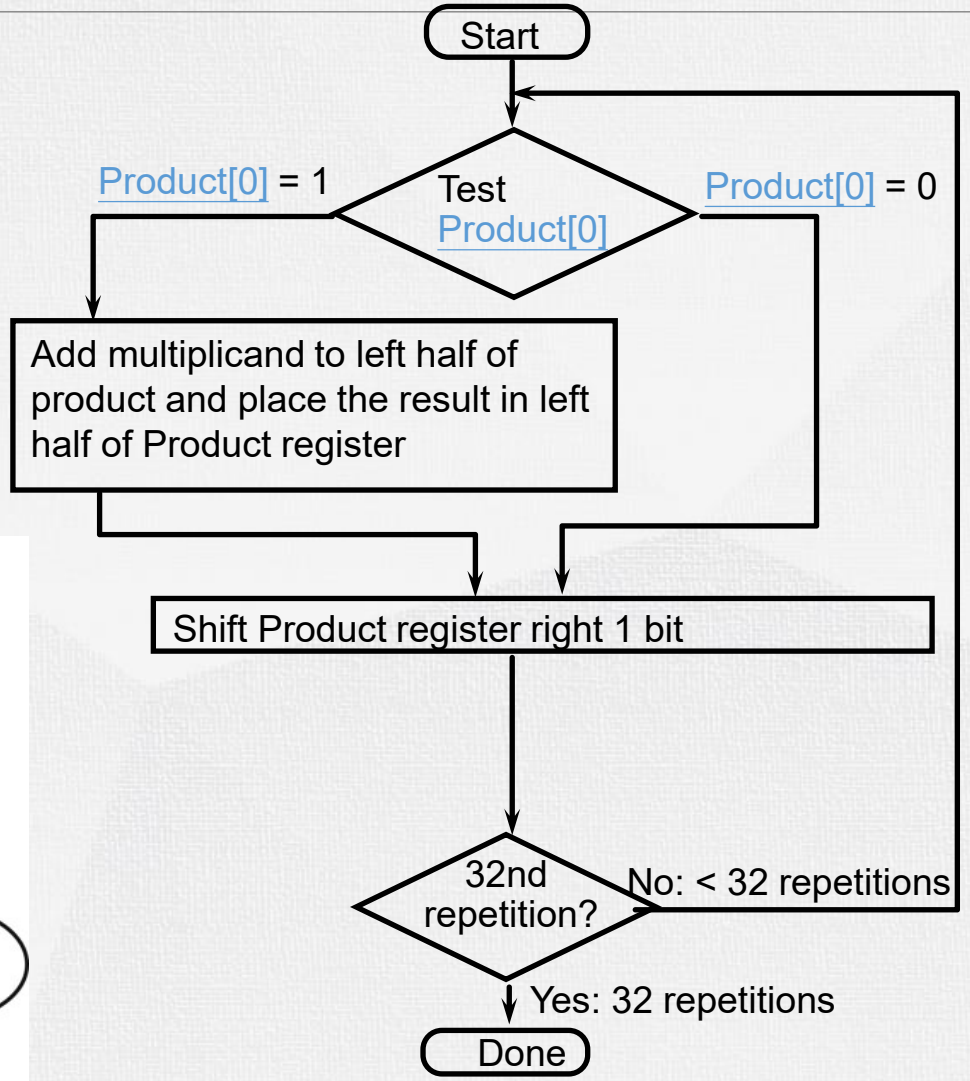
- Multiply 8_{ten} (1000_{two}) by 9_{ten} (1001_{two})

Iter	Step	Multiplier	Multiplicand	Product
0	Initial values	1001	0000 1000	0000 0000
1	1 ⇒ Prod = Prod + Mcand	1001	0000 1000	0000 1000
	Shift left Multiplicand	1001	0001 0000	0000 1000
	Shift right Multiplier	0100	0001 0000	0000 1000
2	0 ⇒ No operation	0100	0001 0000	0000 1000
	Shift left Multiplicand	0100	0010 0000	0000 1000
	Shift right Multiplier	0010	0010 0000	0000 1000
3	Same steps as 2	0010	0010 0000	0000 1000
		0010	0100 0000	0000 1000
		0001	0100 0000	0000 1000
4	Same steps as 1	0001	0100 0000	0100 1000
		0001	1000 0000	0100 1000
		0000	1000 0000	0100 1000

Optimized Multiplier Hardware

- Multiplier initially in right half of product register, 32-bit ALU and multiplicand is untouched
- Check the 0th bit in Product register, if 1, add left half of product with multiplicand
- The sum keeps shifting right, at every step, number of bits in product + multiplier = 64

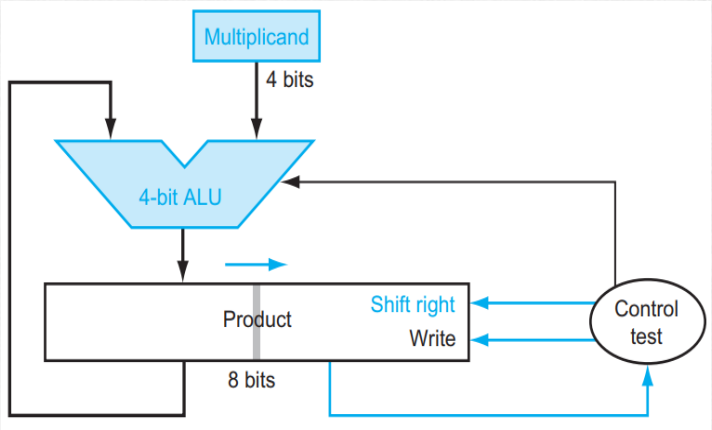




Optimized Multiplier

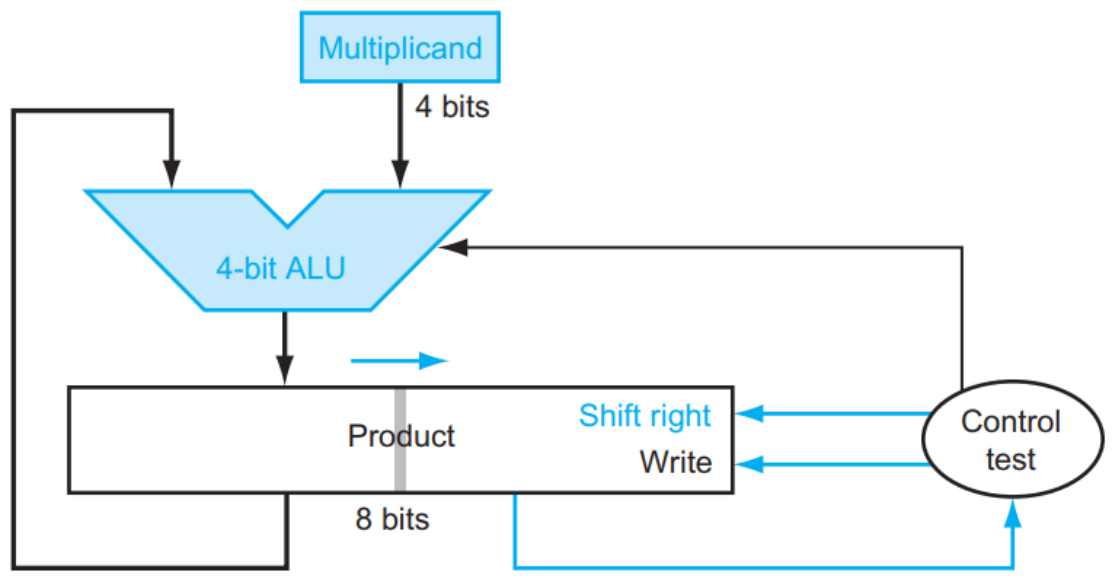
- Example:
  - Multiply 2_{ten} (0010_{two}) by 7_{ten} (0111_{two})
  - result = 00001110_{two} (14_{ten})

iter	Multiplicand	Product	Operation
ini	0010	0000 0111	
1	0010	0010 0111	**1**: Prod left half accumulate Shift right Prod
	0010	0001 0011	
2	0010	0011 0011	**1**: Prod left half accumulate Shift right Prod
	0010	0001 1001	
3	0010	0011 1001	**1**: Prod left half accumulate Shift right Prod
	0010	0001 1100	
4	0010	0000 1110	**0**: Shift right Prod
		res=00001110	done

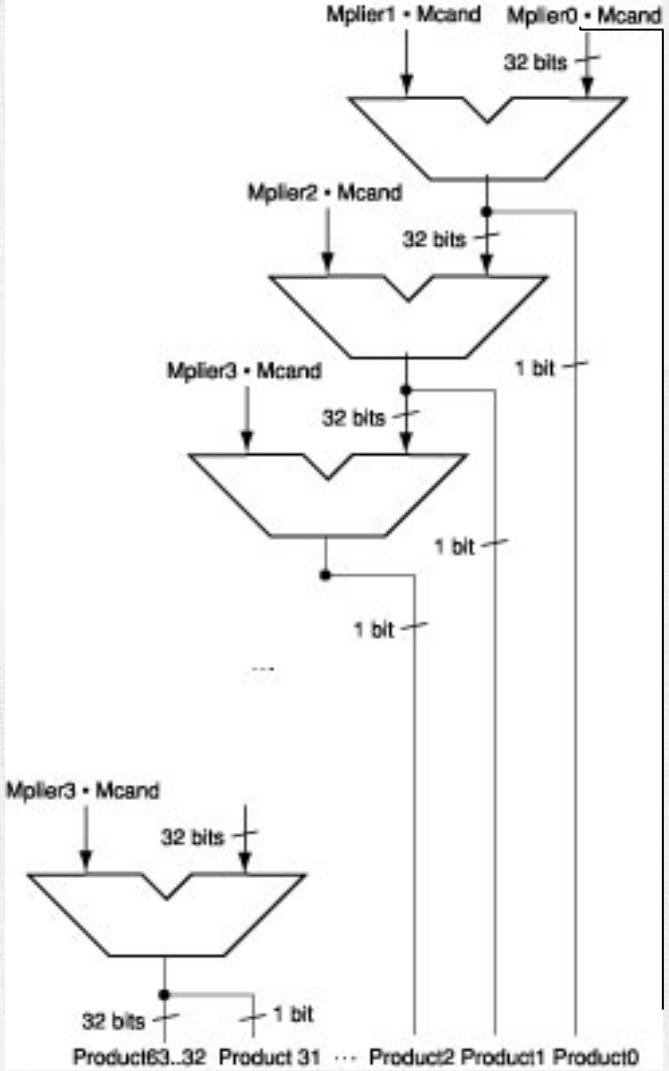


Optimized Multiplier

- Exercise: Multiply 8_{ten} (1000_{two}) by 11_{ten} (1011_{two}) :
- Values in Product register?
- How about Multiplicand register?



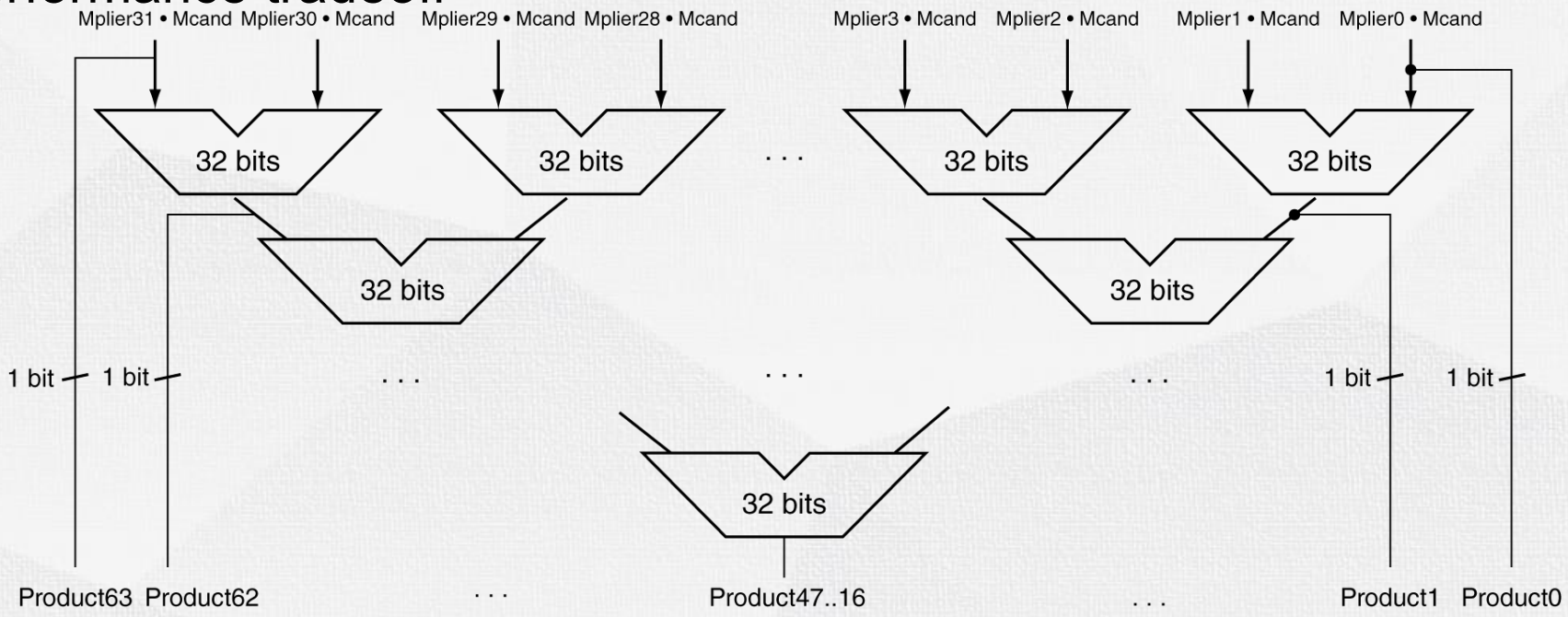
Faster Multiplier



- The previous algorithm requires a clock to ensure that the earlier addition has completed before shifting
- This algorithm can quickly set up most inputs – it then has to wait for the result of each add to propagate down – faster because no clock is involved
- High transistor cost

Faster Multiplier

- Uses multiple pipelined adders
  - Cost/performance tradeoff



- Can be pipelined
  - Several multiplication performed in parallel

Multiplication for Signed Integers

- Previously, we assume both the multiplicand and multiplier are positive integers. How about the multiplication for signed integers?
- Procedure: convert to positive number, calculate the product, then change the sign
- E.g. -7 ✕ -6
- |-7|=7: 0111_2, |-6|=6: 0110_2, 7 ✕ 6 = 0111_2 ✕ 0110_2 = 101010_2 = 42
- So, -7 ✕ -6 = 42
- Be noted that 4-bit multiplicand and multiplier can both represent an unsigned number in range [-8, 7] and a signed number in range [0, 15]
- It can also be calculated by thinking -7 as 11111001 and -6 as 11111010 and extend the sign of the product for signed numbers when we do shifting during the multiplication.

RISC-V Multiplication

- Four multiply instructions:
  - mul: multiply
    - Gives the lower 32 bits of the product
  - mulh: multiply high
    - Gives the upper 32 bits of the product, assuming the operands are signed
  - mulhu: multiply high unsigned
    - Gives the upper 32 bits of the product, assuming the operands are unsigned
  - mulhsu: multiply high signed/unsigned
    - Gives the upper 32 bits of the product, assuming one operand is signed and the other unsigned

Integer Division

Division

quotient

dividend

```
          1001
1000 )1001010
     -1000
          10    (10−1000<0)
          101
          1010
         -1000
           10
```

divisor

remainder

n-bit operands yield n-bit quotient and remainder

- Check for 0 divisor
- Long division approach
  - If divisor ≤ dividend bits
    - 1 bit in quotient, subtract
  - Otherwise
    - 0 bit in quotient, bring down next dividend bit
- Restoring division
  - Do the subtract, and if remainder goes < 0, add divisor back
- Signed division
  - Divide using absolute values
  - Adjust sign of quotient and remainder as required

Division Hardware

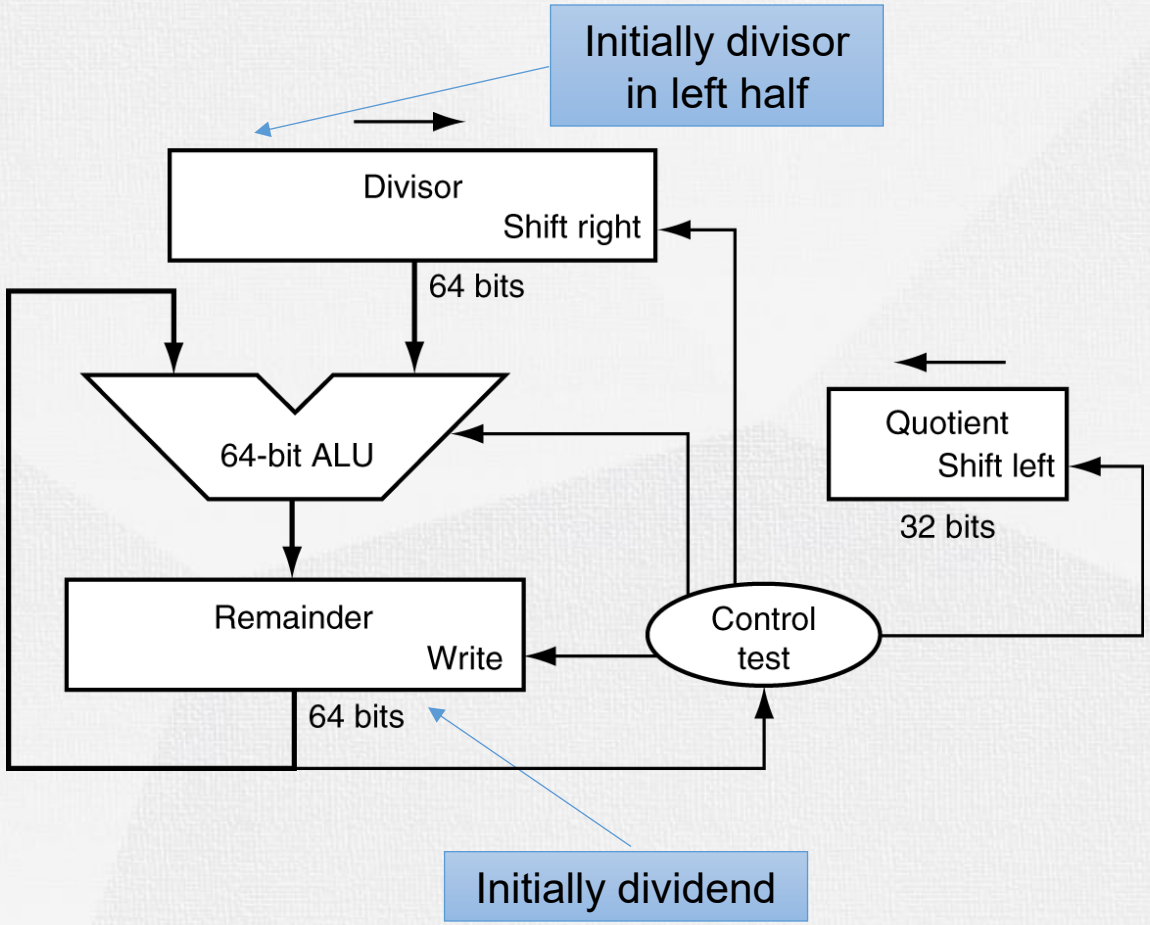
quotient

dividend

divisor

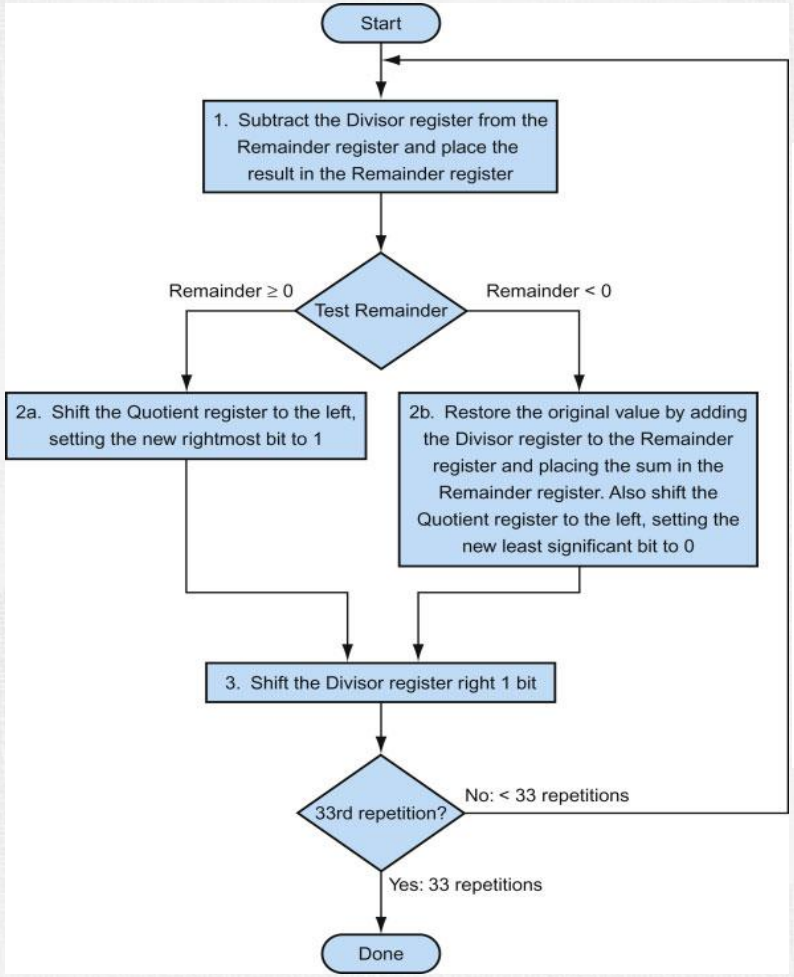
remainder

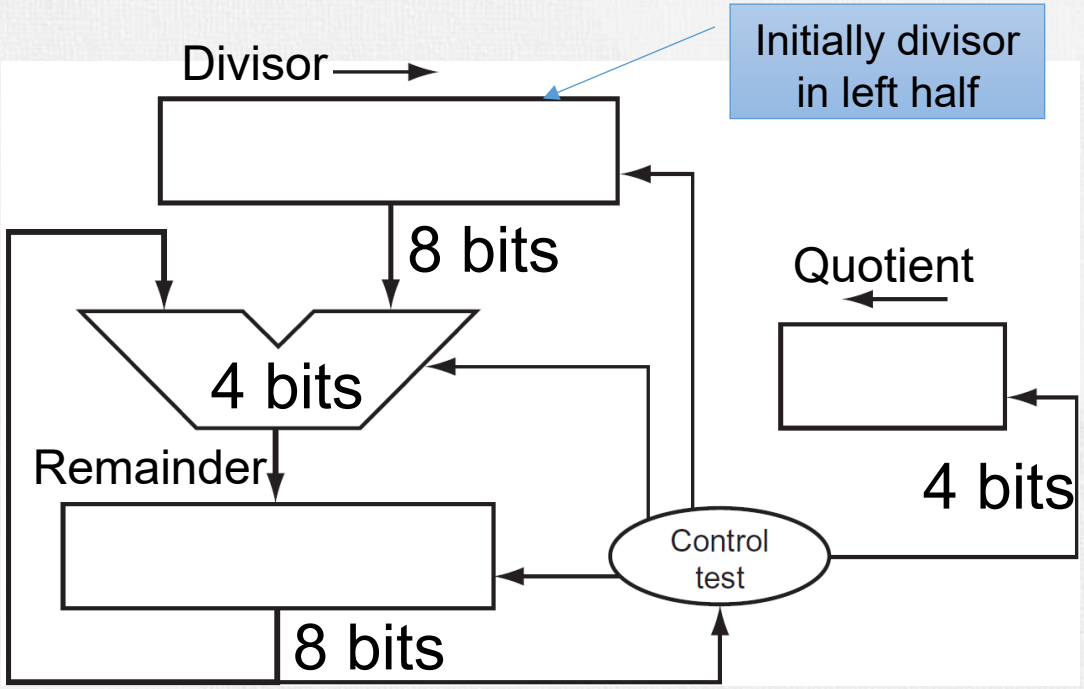
```
        1001
1000 )1001010
     -1000
        10
        101
        1010
       -1000
          10
```



Division Example

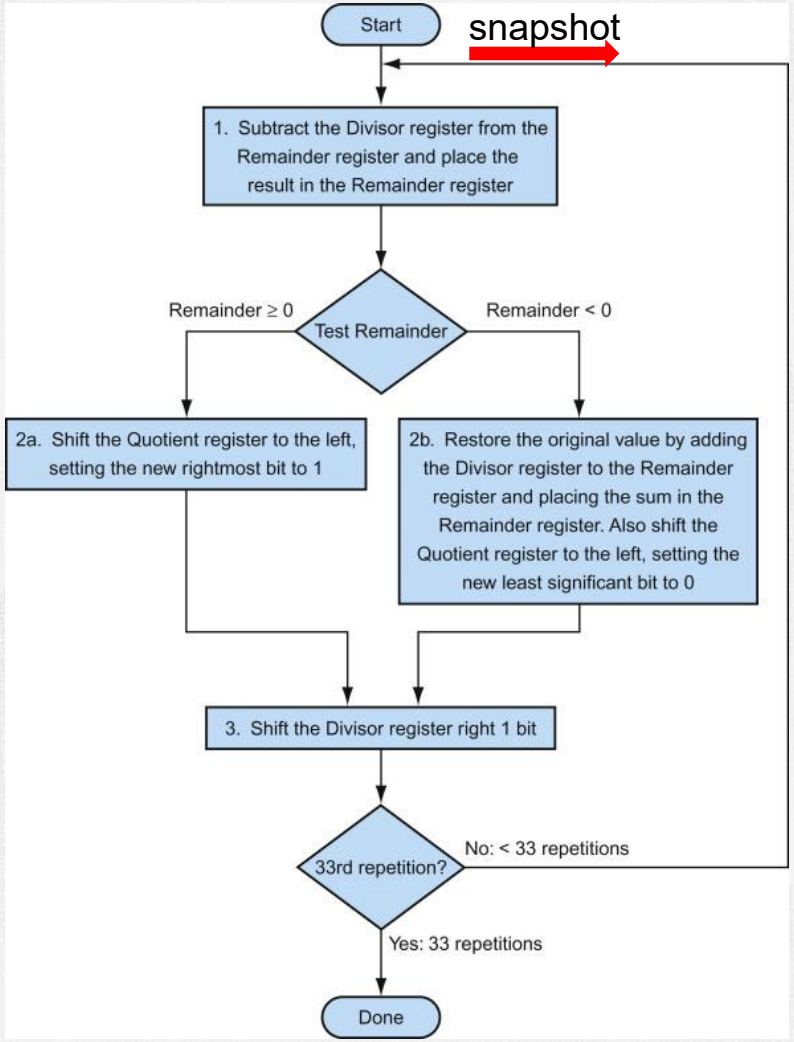
- Divide 7_{ten} ($0000\ 0111_{two}$) by 2_{ten} (0010_{two})

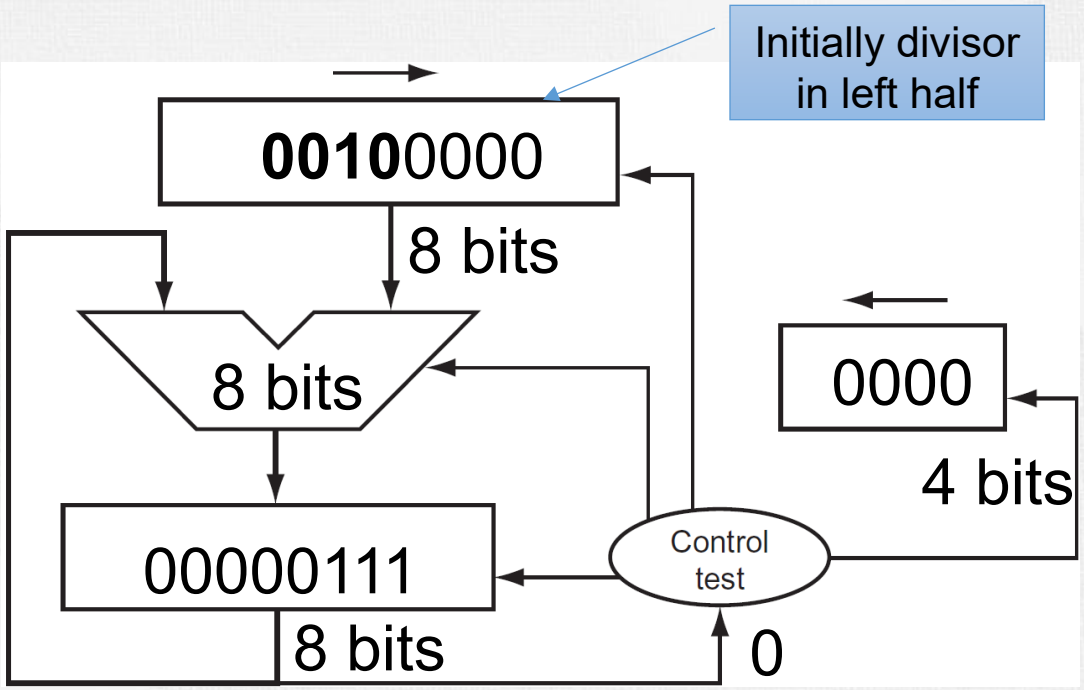




Division Example

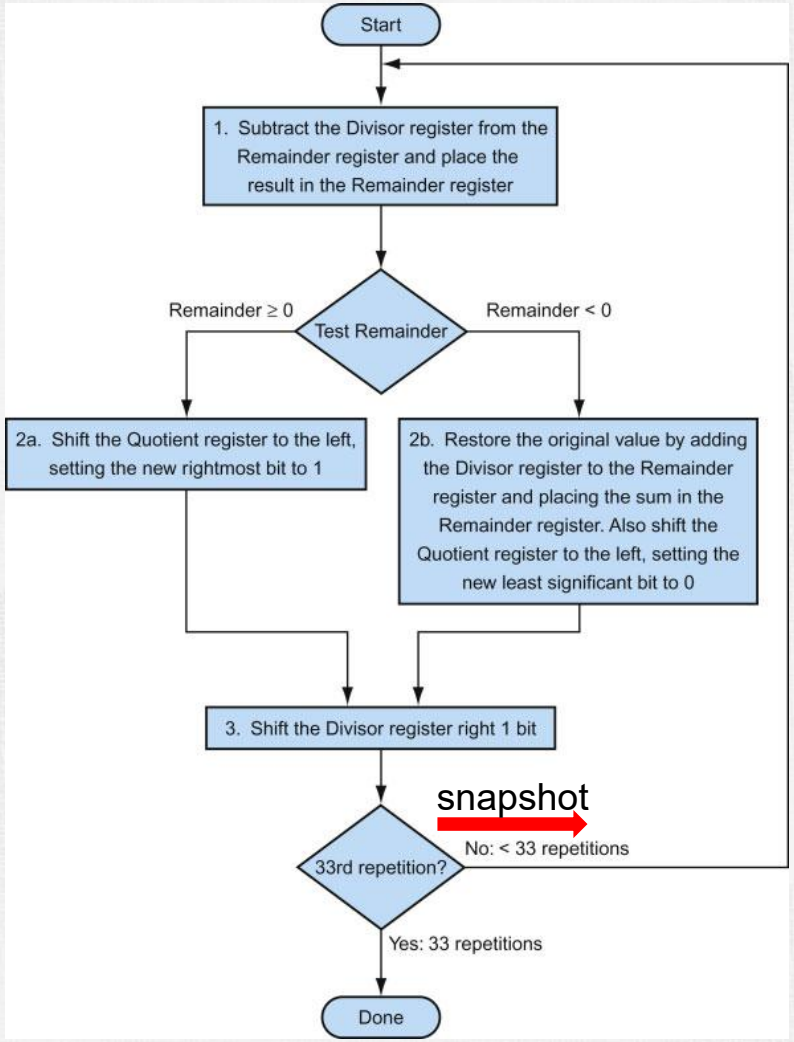
- Divide 7_{ten} ($0000\ 0111_{two}$) by 2_{ten} (0010_{two})

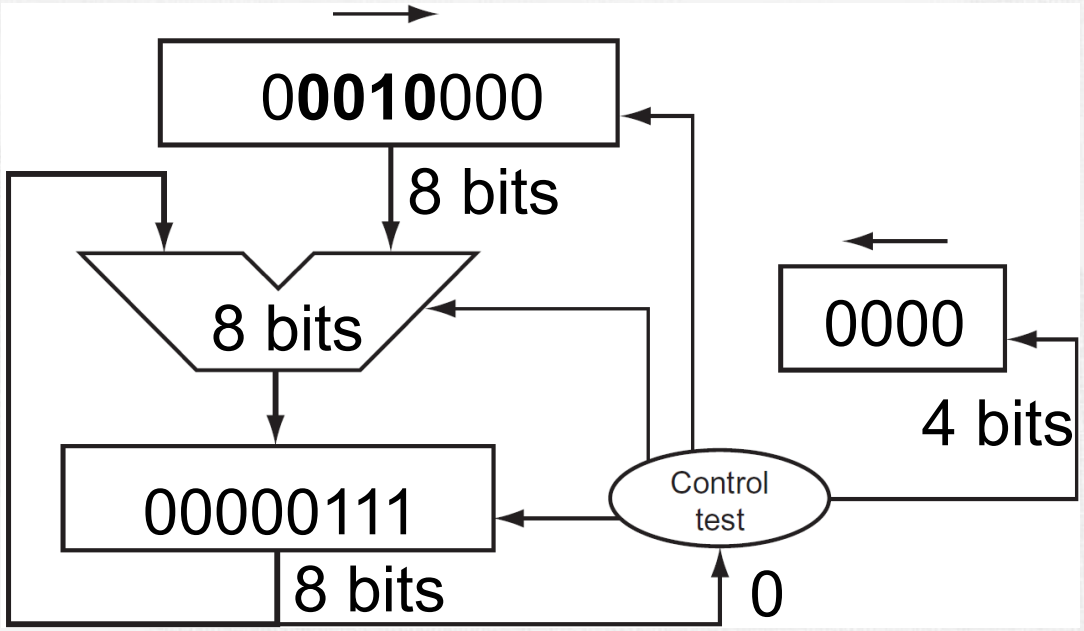




Division Example

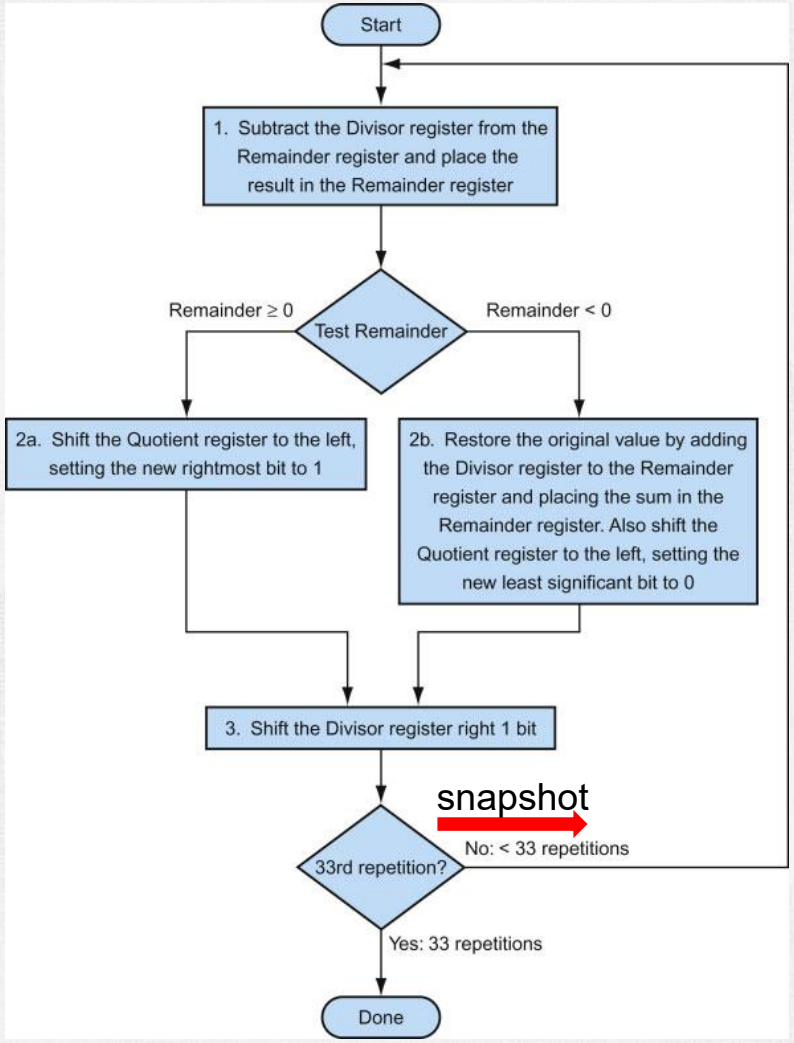
- Divide 7_{ten} ($0000\ 0111_{two}$) by 2_{ten} (0010_{two})

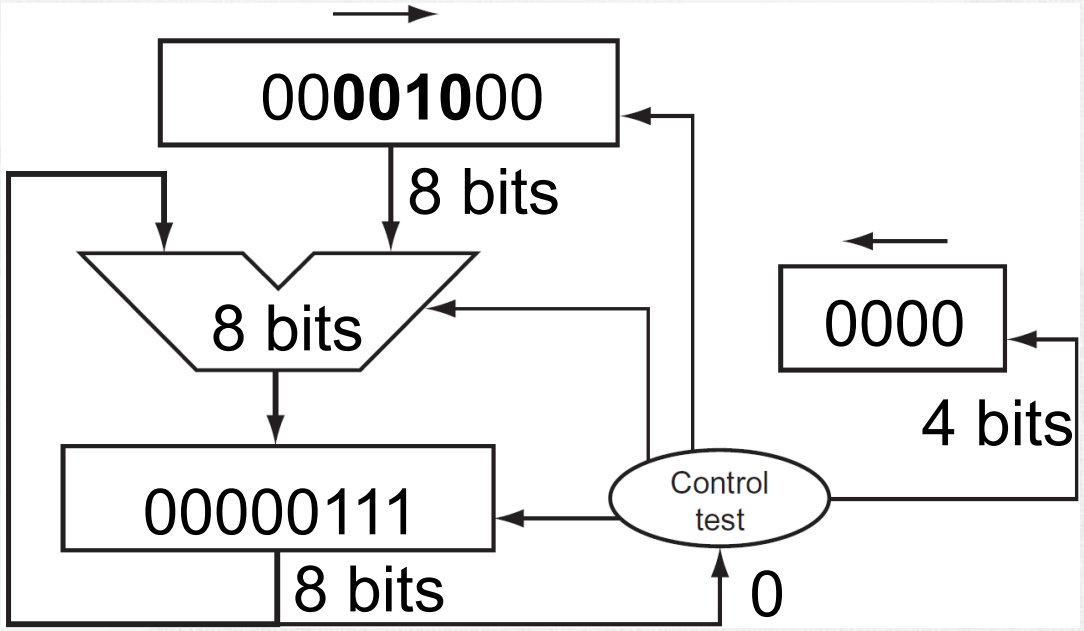




Division Example

- Divide 7_{ten} ($0000\ 0111_{two}$) by 2_{ten} (0010_{two})

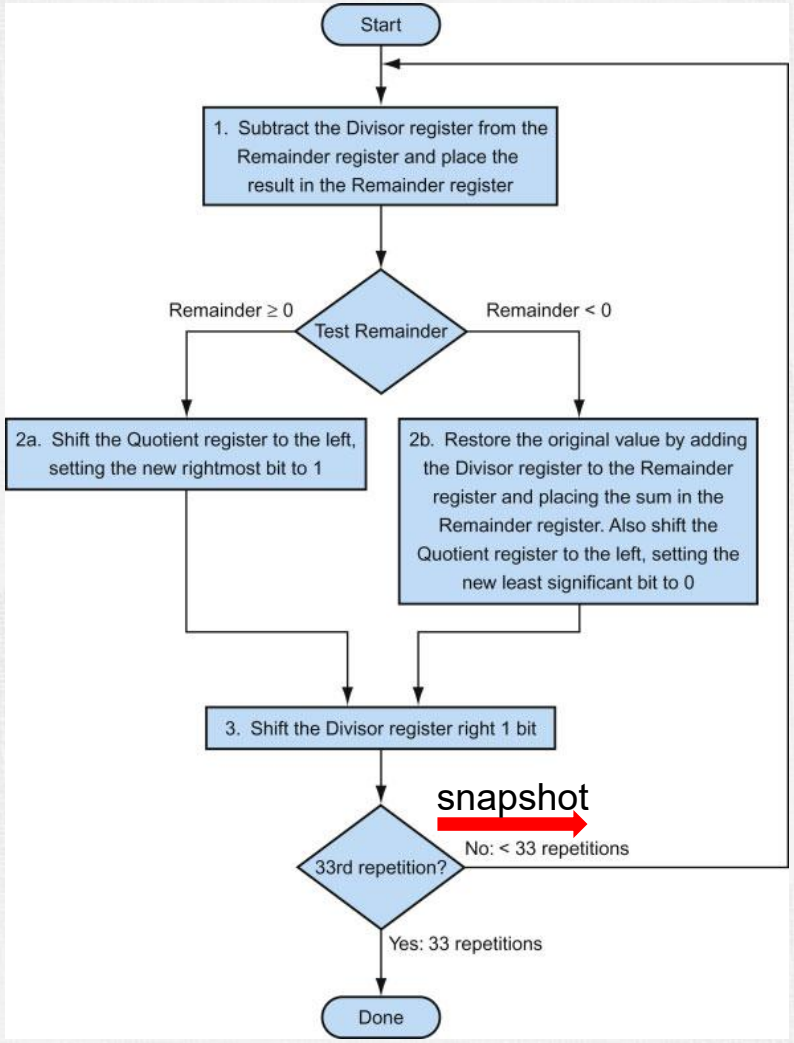


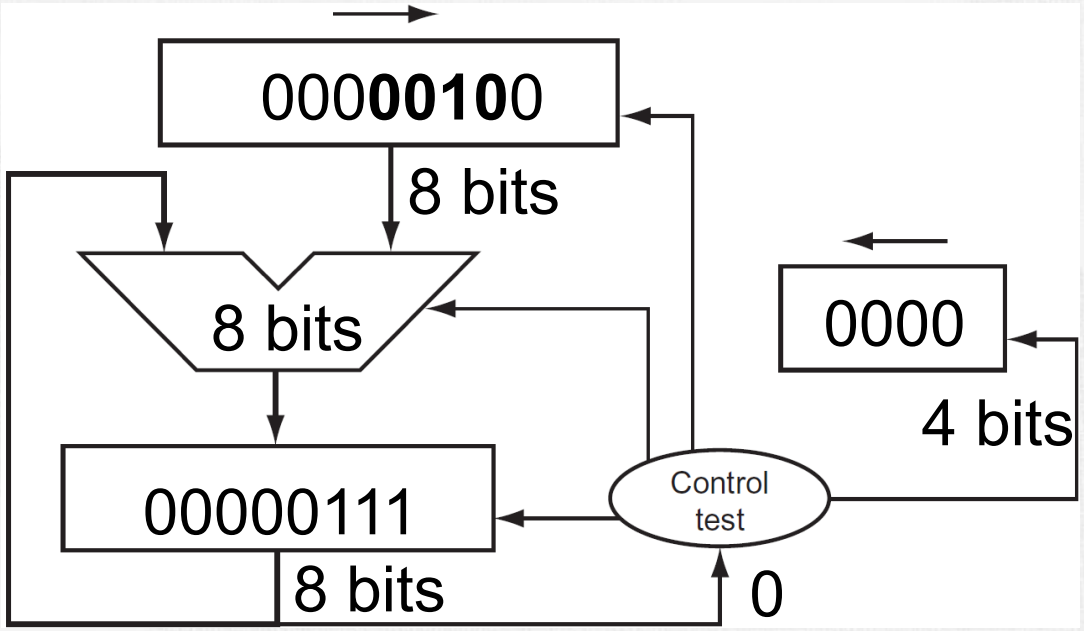


zhangj4@sustech.edu.cn

Division Example

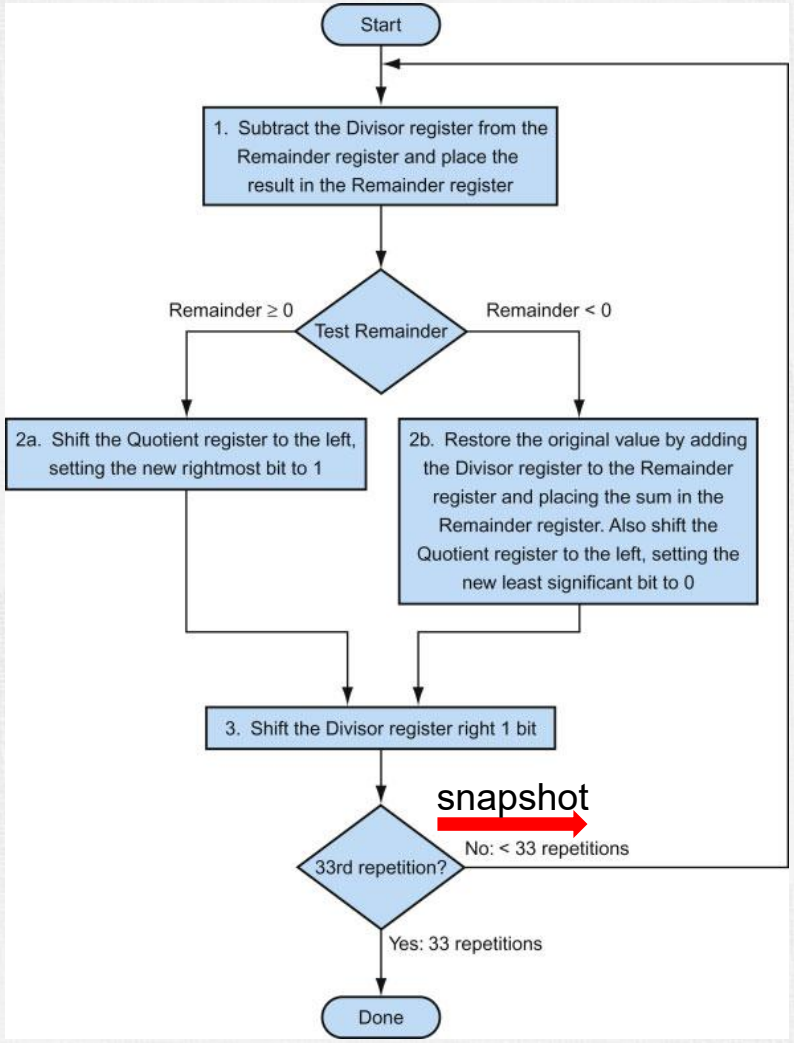
- Divide 7_{ten} ($0000\ 0111_{two}$) by 2_{ten} (0010_{two})

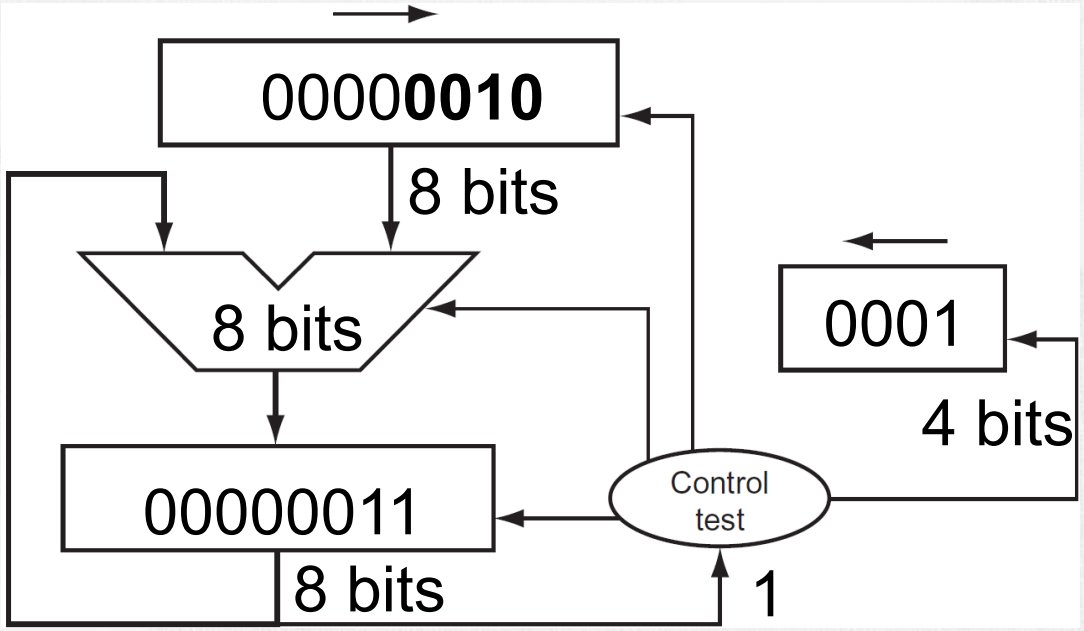




Division Example

- Divide 7_{ten} ($0000\ 0111_{two}$) by 2_{ten} (0010_{two})

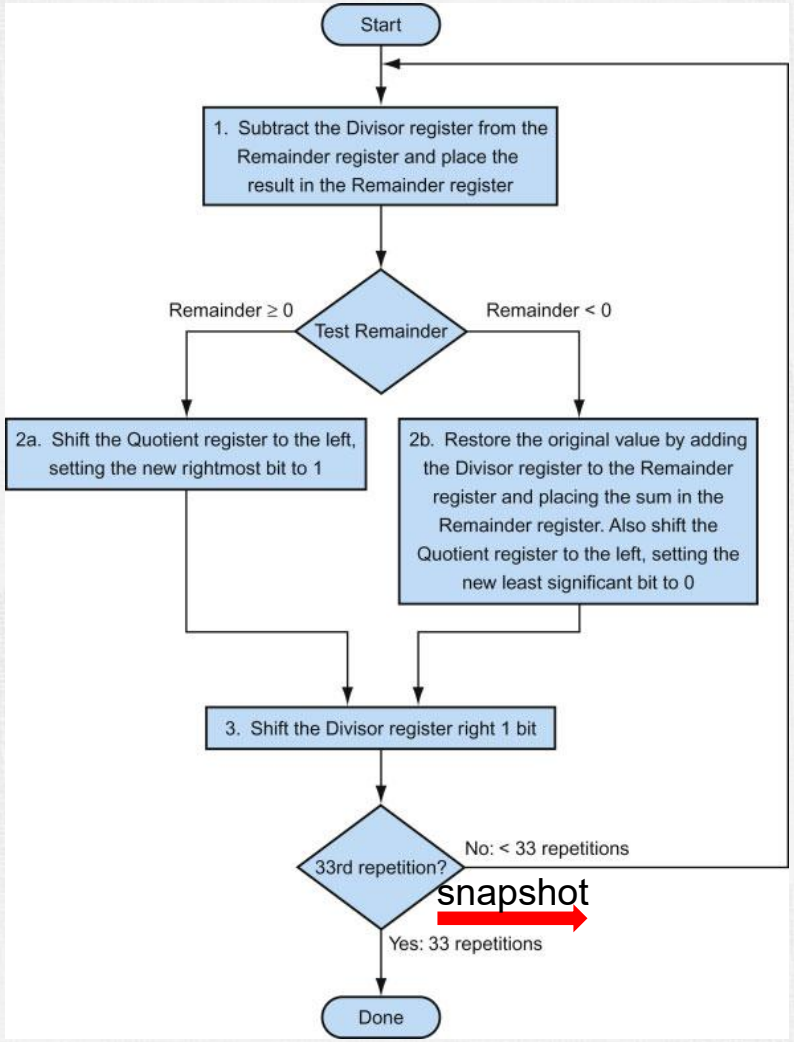




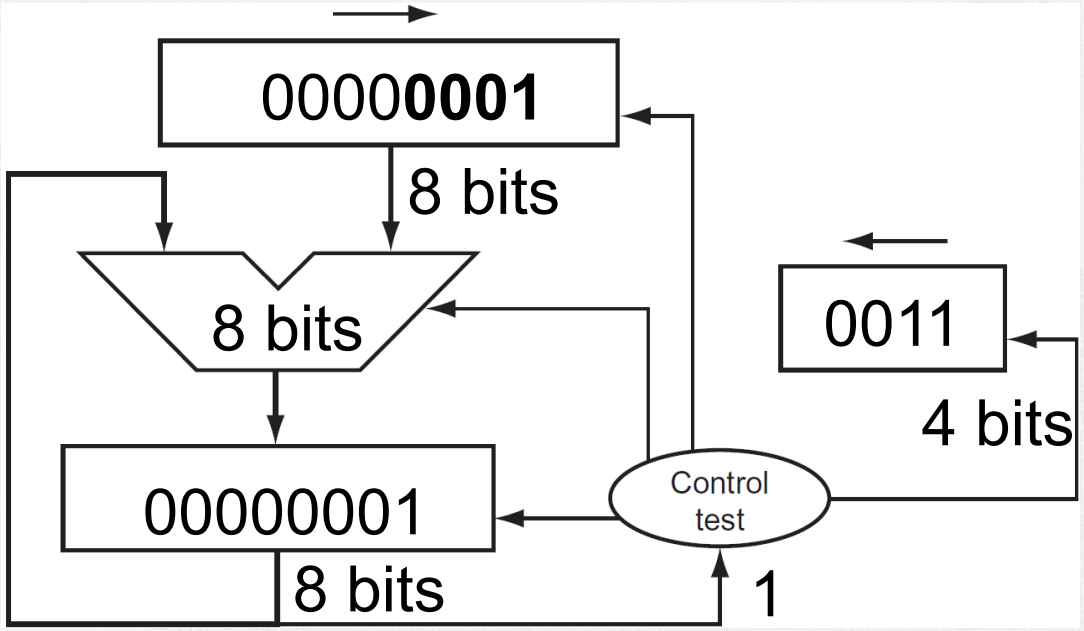
zhangj4@sustech.edu.cn

Division Example

- Divide 7_{ten} ($0000\ 0111_{two}$) by 2_{ten} (0010_{two})



snapshot



Done!

Division Example

Divide 7_{ten} ($0000\ 0111_{two}$) by 2_{ten} (0010_{two})

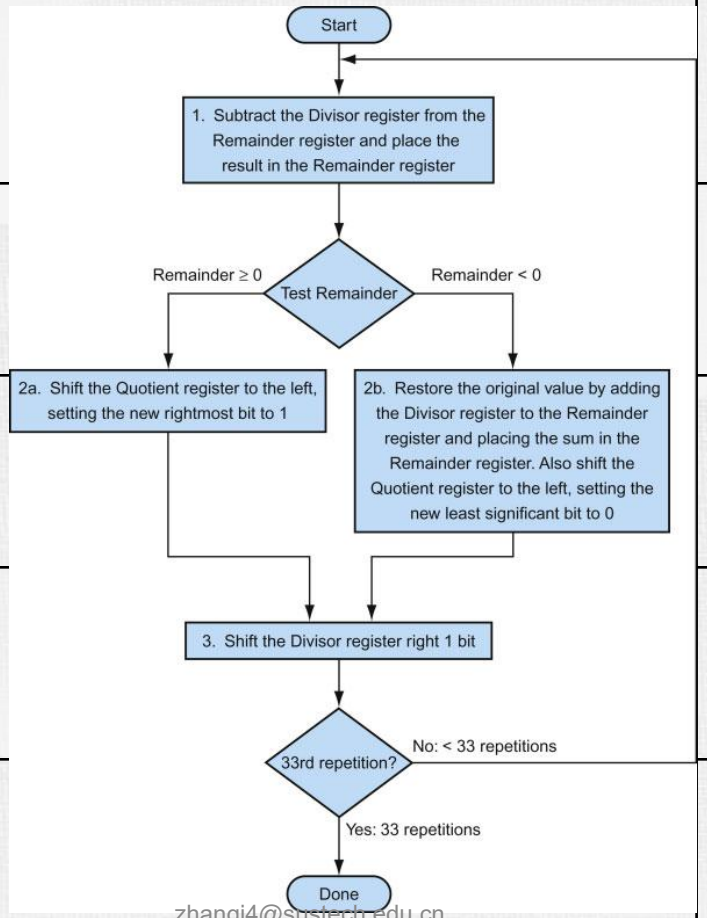
Iter	Step	Quot	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	Rem = Rem – Div Rem < 0 ➔ +Div, shift 0 into Q Shift Div right	0000 0000 0000	0010 0000 0010 0000 0001 0000	1110 0111 0000 0111 0000 0111
2	Same steps as 1	0000 0000 0000	0001 0000 0001 0000 0000 1000	1111 0111 0000 0111 0000 0111
3	Same steps as 1	0000 0000 0000	0000 1000 0000 1000 0000 0100	1111 1111 0000 0111 00000111
4	Rem = Rem – Div Rem >= 0 ➔ shift 1 into Q Shift Div right	0000 0001 0001	0000 0100 0000 0100 0000 0010	0000 0011 0000 0011 0000 0011
5	Same steps as 4	0001 0011 0011	0000 0010 0000 0010 0000 0001	0000 0001 0000 0001 0000 0001

8-bit dividend/4-bit divisor

Divide 74_{ten} (1001010_{two}) by 8_{ten} (1000_{two})

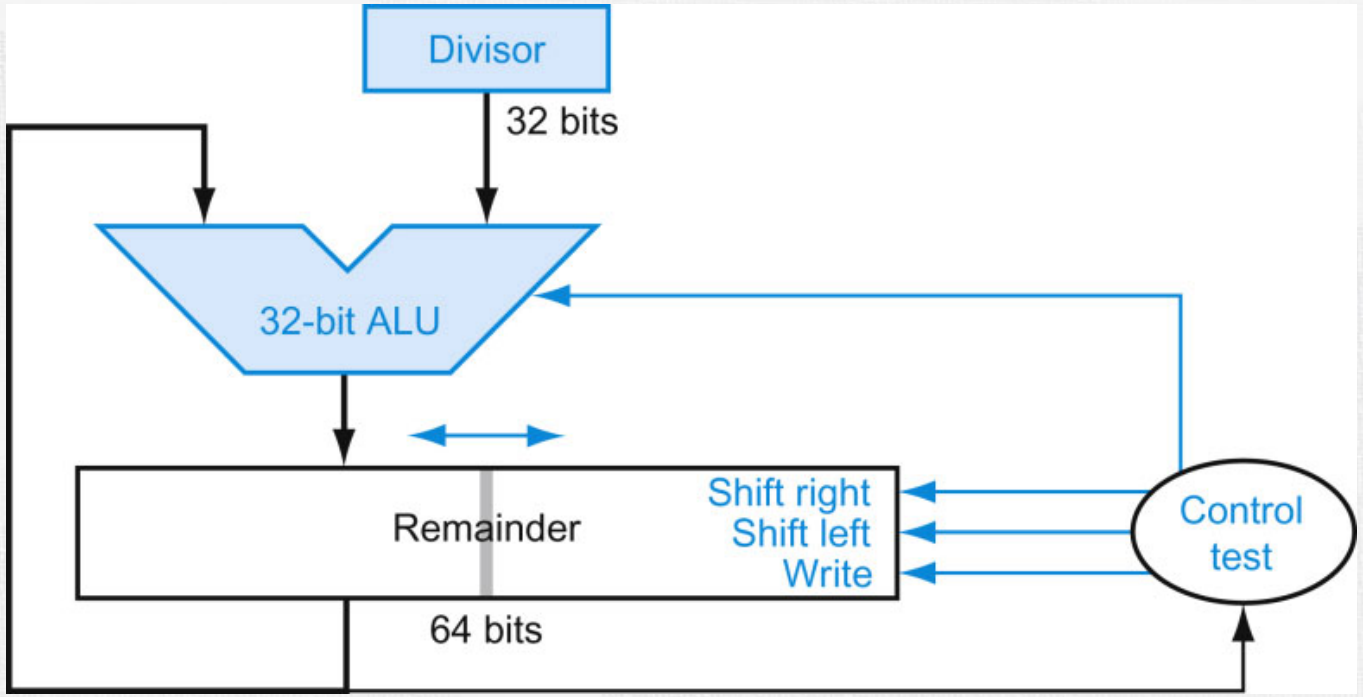
```
          1001
1000 )1001010
     -1000
        10
        101
        1010
       -1000
          10
```

Iter	Step	Quot	Divisor	Remainder
0	Initial values	0000	10000000	01001010
1	1.1 1.2b 1.3	0000 0000 0000	10000000 10000000 01000000	11001010 01001010 01001010
2	2.1 2.2a 2.3	0000 0001 0001	01000000 01000000 00100000	00001010 00001010 00001010
3	3.1 3.2b 3.3	0001 0010 0010	00100000 00100000 00010000	11100101 00001010 00001010
4	4.1 4.2b 4.3	0010 0100 0100	00010000 00010000 00001000	11111010 00001010 00001010
5	5.1 5.2b 5.3	0100 1001 1001	00001000 00001000 00000100	00000010 00000010 00000010



Optimized Divider

- One cycle per partial-remainder subtraction
- Looks a lot like a multiplier!
  - Same hardware can be used for both



Signed Division

- Convert to positive and adjust sign later
- Note that multiple solutions exist for the equation:

  Dividend = Quotient x Divisor + Remainder

+7	div	+2	Quo = +3	Rem = +1
-7	div	+2	Quo = -3	Rem = -1

Why not -7 div +2 Quo = -4 Rem = +1?
If so, -(x div y) != (-x) div y → programming challenge!

+7	div	-2	Quo = -3	Rem = +1
-7	div	-2	Quo = +3	Rem = -1

- Convention:
  - Dividend and remainder have the same sign
  - Quotient is negative if signs disagree
  - These rules fulfill the equation above

Faster Division

- Can’t use parallel hardware as in multiplier
  - Subtraction is conditional on sign of remainder
- Faster dividers (e.g. SRT devision) generate multiple quotient bits per step
  - Still require multiple steps

RISC-V Division

- Four instructions:
  - div, rem: signed divide, remainder
  - divu, remu: unsigned divide, remainder

- Overflow and division-by-zero don’t produce errors
  - Just return defined results
  - Faster for the common case of no error